

LINK:

report: <https://drive.google.com/drive/folders/1wyFyrsnkq0EP29wwjDVQu2TsvfK4lcR8?usp=sharing>

source code: https://drive.google.com/drive/u/0/folders/1VGw4wGYq8Ja_X3K6BORQ_0qu13h73ty6

retrospective video link :

https://www.youtube.com/watch?v=SbcEoN1emZA&list=PLQ4NSqK_9FJCsgwseYZwNo8bK-wD9_Vn&pp=sAgC

slide: <https://canva.link/uozaebk2k636yn6>

website: <https://fourkon.onrender.com/>



รายงานการออกเว็บ star wish

จัดทำโดย

นาย ณัฐชนน มลาสินธ์ 67102010163

นาย เกียรติยศ ม่วงเปลี่ยน 67102010508

นาย ณัฏพล โพธิ์ตาตทอง 67102010512

นาย ณัฐวุฒิ โตเชื้อ 67102010513

เสนอ

อาจารย์ ดร. วีรยุทธ เจริญเรืองกิจ

รายงานนี้เป็นส่วนประกอบของวิชา software engineer

สาขาวิทยาการคอมพิวเตอร์ มหาวิทยาลัยศรีนครินทรวิโรฒ

ปีการศึกษา พ.ศ.2569

สารบัญ

ที่มาของปัญหาและความสำคัญ	3
จุดประสงค์ของโครงการ	4
ขอบเขตของโครงการ	4
Use-case diagram	7
Development Process	16
1. การทำงานของโปรแกรม (Program Workflow)	17
รายละเอียด routes หลัก	17
โครงสร้าง Partial (Reusable Components)	21
1. การเรียกใช้งาน API และการประมวลผลข้อมูล	21
3. ตารางทดสอบ Class และ Module อื่นๆ	32
3. Test Case Code	33
4. Test Coverage Report	54
กระบวนการพัฒนา Software ของโครงการ 4kon	89
กระบวนการทำงานใน Sprint ที่ 1	89
กระบวนการทำงานใน Sprint ที่ 2	90
กระบวนการทำงานใน Sprint ที่ 3	90
การพัฒนาและเขียนโค้ด (Implementation)	90
การทดสอบระบบ (Testing)	91
กระบวนการทำงานใน Sprint ที่ 4	91
สิ่งที่มีการเปลี่ยนแปลงจาก Sprint ที่ 1	91
สิ่งที่มีการเปลี่ยนแปลงจาก Sprint ที่ 2	92
สิ่งที่มีการเปลี่ยนแปลงจาก Sprint ที่ 3	92

ที่มาของปัญหาและความสำคัญ

ในปัจจุบันการซื้อขายสินค้าออนไลน์กลายเป็นพฤติกรรมปกติของผู้คน เนื่องจากมีความสะดวก รวดเร็ว และมีตัวเลือกสินค้าที่หลากหลายจากหลายแพลตฟอร์ม เช่น เว็บไซต์อีคอมเมิร์ซและร้านค้าออนไลน์ต่าง ๆ อย่างไรก็ตาม ผู้ใช้งานมักพบปัญหาในการจัดการข้อมูลสินค้าที่ตนเองสนใจ ไม่ว่าจะเป็นการจํารายละเอียดสินค้า ราคา ลิงก์ หรือการเปรียบเทียบสินค้าหลายรายการในเวลาเดียวกัน หลายครั้งผู้ใช้งานเห็นสินค้าแล้วต้องการซื้อในภายหลัง แต่ไม่สามารถเก็บข้อมูลไว้ได้อย่างเป็นระบบ ทำให้เกิดปัญหาการลืมสินค้า ลืมราคา หรือเสียเวลาในการค้นหาใหม่อีกครั้ง นอกจากนี้ การวางแผนการใช้จ่ายเพื่อซื้อสินค้ายังเป็นเรื่องสำคัญ โดยเฉพาะในกลุ่มนักเรียน นักศึกษา หรือผู้ที่มีงบประมาณจำกัด หากไม่มีเครื่องมือช่วยจัดการรายการสินค้าที่อยากซื้อ ก็อาจทำให้เกิดการใช้จ่ายเกินความจำเป็น หรือไม่สามารถจัดลำดับความสำคัญของสิ่งที่ต้องการได้อย่างเหมาะสม ดังนั้น จึงเกิดแนวคิดในการพัฒนาเว็บแอปพลิเคชันที่ช่วยให้ผู้ใช้งานสามารถบันทึกและจัดการข้อมูลสินค้าที่ตนเองสนใจได้อย่างเป็นระบบ โดยรวบรวมรายละเอียดสำคัญของสินค้าไว้ในที่เดียว เพื่ออำนวยความสะดวกในการวางแผนการซื้อและช่วยให้การตัดสินใจมีประสิทธิภาพมากขึ้น

จุดประสงค์ของโครงการ

โครงการนี้มีจุดประสงค์เพื่อพัฒนาเว็บแอปพลิเคชันสำหรับช่วยผู้ใช้งานในการจัดการรายการสินค้าที่ต้องการซื้อ โดยผู้ใช้งานสามารถเพิ่มข้อมูลสินค้าได้ด้วยตนเอง เช่น รูปภาพสินค้า ราคา ประเภทสินค้า และลิงก์แหล่งที่มา ระบบจะช่วยรวบรวมข้อมูลเหล่านี้ไว้ในรูปแบบที่เป็น

เปรียบเทียบง่ายต่อการค้นหาและเปรียบเทียบ อีกทั้งโครงการยังมุ่งเน้นให้ผู้ใช้สามารถวางแผนการใช้จ่ายได้อย่างมีประสิทธิภาพ ผ่านการดูรายการสินค้าทั้งหมดที่สนใจ ซึ่งช่วยให้ผู้ใช้มองเห็นภาพรวมของค่าใช้จ่ายที่อาจเกิดขึ้นในอนาคต นำไปสู่การตัดสินใจซื้ออย่างรอบคอบ ลดการซื้อแบบฉาบฉวย และส่งเสริมการบริหารเงินส่วนบุคคล นอกจากนี้ โครงการยังเป็นการฝึกกระบวนการพัฒนาซอฟต์แวร์แบบเป็นขั้นตอน ตั้งแต่การเก็บความต้องการของผู้ใช้ การวิเคราะห์และออกแบบระบบ การทำงานร่วมกันเป็นทีม ไปจนถึงการใช้เครื่องมือจัดการโครงการ เช่น GitHub ในการติดตามงานและควบคุมเวอร์ชัน ซึ่งช่วยเสริมสร้างทักษะที่จำเป็นสำหรับการพัฒนาระบบจริงในอนาคต

ขอบเขตของโครงการ

ขอบเขตด้านผู้ใช้งาน โดยผู้ใช้งานทั่วไปสามารถ:

1. ดูรายการสินค้าจาก Shopee และ Lazada
2. แยกดูสินค้าเป็นหมวดหมู่ (Category)
3. กรองแหล่งที่มาของสินค้า เช่น แสดงเฉพาะ Shopee หรือ Lazada
4. เรียงลำดับราคาสินค้า (จากน้อยไปมาก / มากไปน้อย)
5. ดูสินค้าโปรโมชั่น (Promotion)
6. เพื่อดูหน้า About Us เพื่อดูข้อมูลเกี่ยวกับบริษัท/ทีมพัฒนา

ขอบเขตด้านผู้ดูแลระบบ โดยผู้ดูแลระบบสามารถ:

1. เพิ่มสินค้าใหม่เข้าสู่ระบบ
2. แก้ไขข้อมูลสินค้า
3. ลบสินค้าออกจากระบบ
4. จัดการหมวดหมู่สินค้า
5. ตรวจสอบ Log การกดซื้อสินค้าจากเว็บไซต์
6. มีหน้าต่าง dash board ทางสถิติเพื่อสามารถเปรียบเทียบข้อมูลต่างๆได้

ขอบเขตด้านแพลตฟอร์ม

ระบบต้องสามารถใช้งานได้ผ่านเว็บเบราว์เซอร์ และรองรับการแสดงผลทั้งบน คอมพิวเตอร์ (PC) และ โทรศัพท์มือถือ (Mobile)

Functional Requirements and NON-Functional Requirements

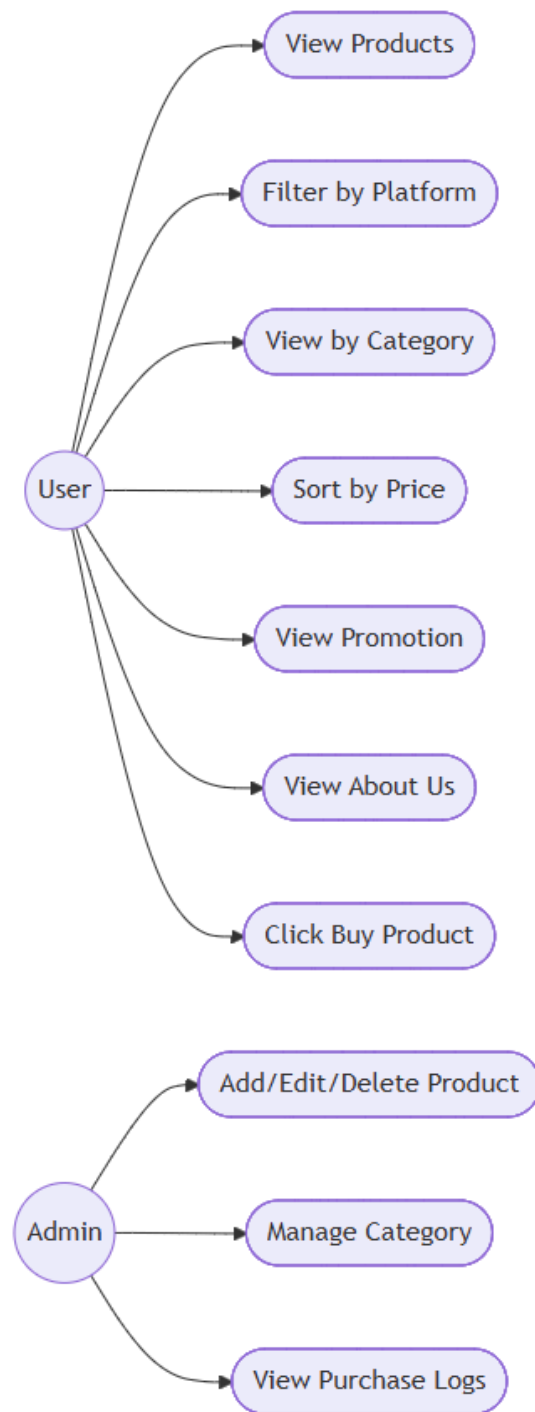
Functional Requirement :

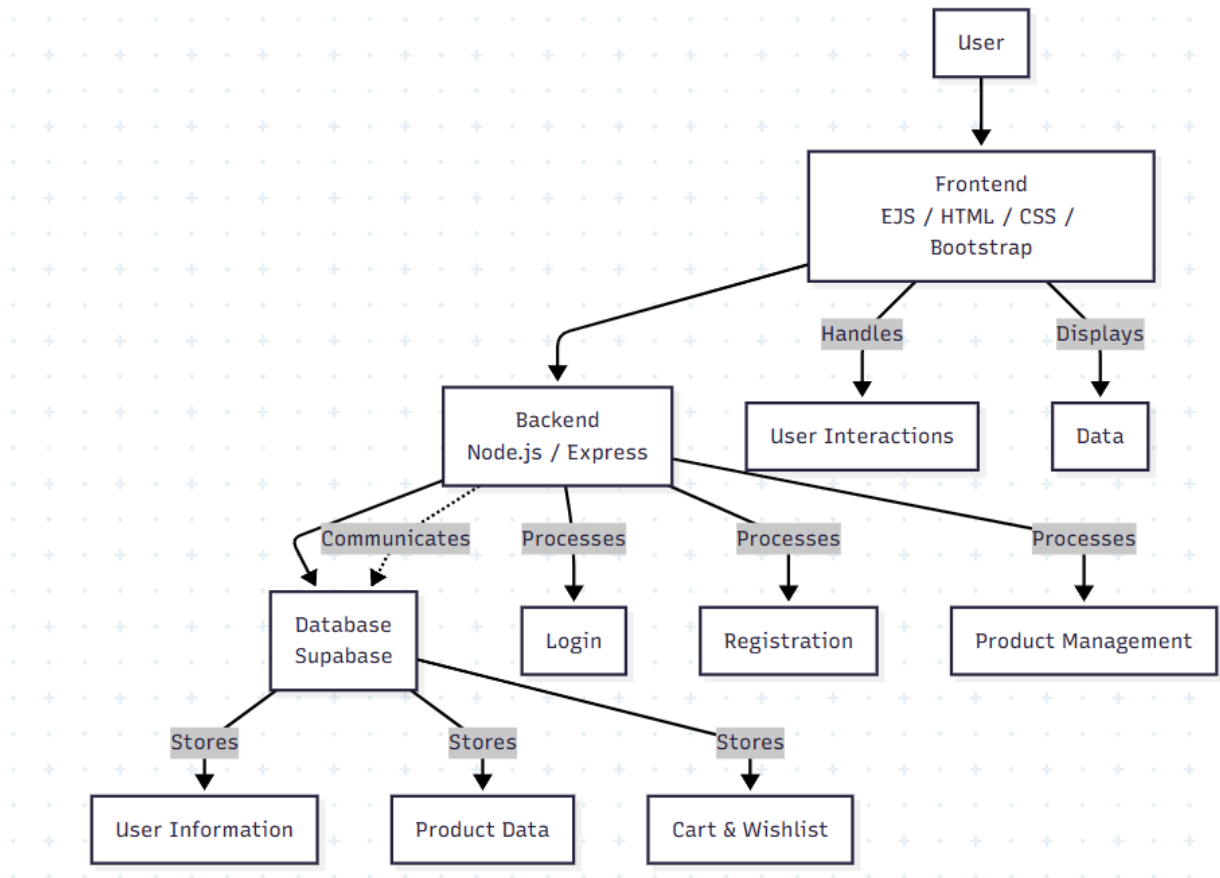
1. ระบบต้องแยกหมวดหมู่สินค้าได้ ผู้ใช้สามารถดูสินค้าแยกตาม Category
2. ระบบต้องกรองแหล่งที่มาของสินค้าได้ ผู้ใช้สามารถเลือกดูเฉพาะสินค้าจาก Shopee หรือ Lazada เท่านั้น
3. ระบบต้องเรียงราคาสินค้าได้ ผู้ใช้สามารถเรียงลำดับราคาจากน้อยไปมาก หรือมากไปน้อย
4. ระบบต้องมีระบบจัดการสินค้า (เพิ่ม/ลบ/แก้ไข) Admin สามารถเพิ่ม ลบ และแก้ไขข้อมูลสินค้าได้
5. ระบบต้องมีการกำหนดบทบาทผู้ใช้ (Role Management) แยกระหว่าง Admin และ User พร้อมสิทธิ์การใช้งานที่แตกต่างกัน
6. ระบบต้องรองรับการแสดงผล Promotion แสดงสินค้าลดราคา หรือโปรโมชั่นพิเศษ
7. ระบบต้องมีหน้า About Us แสดงข้อมูลเกี่ยวกับบริษัทหรือทีมพัฒนา
8. ระบบต้องบันทึก Log การกดซื้อสินค้า Admin สามารถดูประวัติการคลิกซื้อจากเว็บไซต์ได้

Non-Functional Requirement :

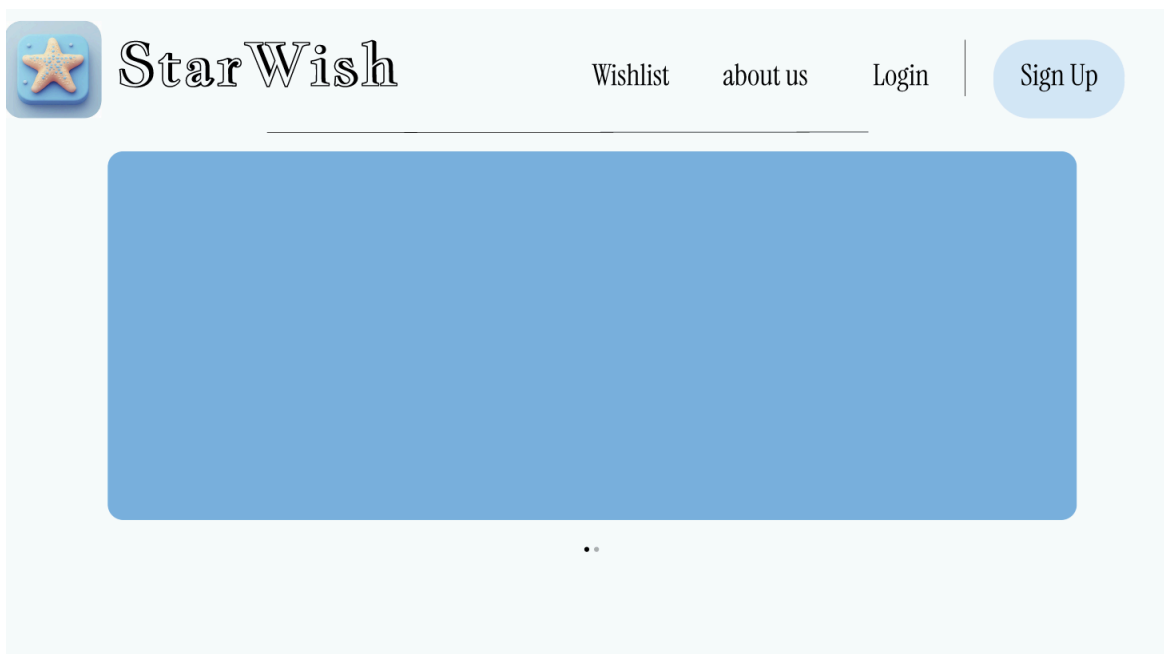
1. ระบบต้องใช้โทสนีหลักเพียงโทสนีเดียวตลอดทั้งแอปพลิเคชัน เช่น โทสนีฟ้า หรือโทสนีเทา-แดง และต้องไม่ใช่หลายสีปนกันเพื่อความสม่ำเสมอของการออกแบบ
2. ระบบต้องสามารถรองรับผู้ใช้งานจำนวนมากพร้อมกันได้ โดยไม่เกิดอาการช้า ค้าง หรือหน่วงในการแสดงผลหรือประมวลผลข้อมูล
3. หน้าเว็บต้องแสดงผลได้อย่างเหมาะสมทั้งบนคอมพิวเตอร์ (PC) และอุปกรณ์พกพา (Mobile) โดยใช้การออกแบบแบบ Responsive Design
4. ระบบต้องออกแบบให้ใช้งานง่าย เมนูชัดเจน เข้าใจได้ทันที ลดความซับซ้อน เพื่อให้ผู้ใช้งานสามารถใช้งานได้โดยไม่ต้องมีคู่มือ
5. ระบบต้องหลีกเลี่ยงการใช้ Animation ที่มากเกินไป เพื่อไม่ให้กระทบต่อความเร็วของระบบ และไม่รบกวนประสบการณ์การใช้งานของผู้ใช้
6. ระบบบันทึก Log การกดซื้อสินค้าต้องทำงานถูกต้องทุกครั้ง และข้อมูลต้องไม่สูญหายแม้เกิดข้อผิดพลาดของระบบบางส่วน
7. โครงสร้างโค้ดของระบบต้องออกแบบให้สามารถแก้ไข ปรับปรุง หรือเพิ่มฟีเจอร์ในอนาคตได้ง่าย เช่น การแยกส่วนการทำงานของระบบอย่างชัดเจน

Use-case diagram





Main Page



REGISTER



Register

[Login](#)

LOGIN



LOGIN

Forget your password?

[Sign up](#)

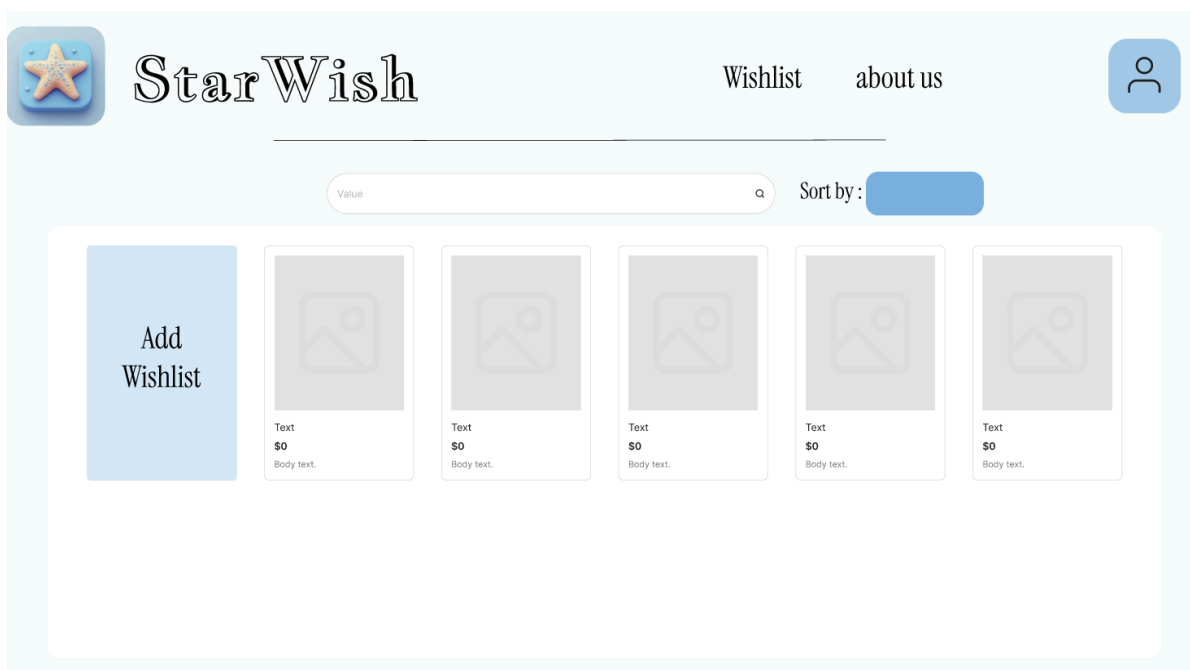
FORGOT PASSWORD



Forgot Password

[Login](#)

WHISTLIST



PRODUCT DETAIL

Item Name:



Insert Picture

category

Description

ABOUT US



StarWish

[Wishlist](#) [about us](#) [Login](#) [Sign Up](#)

About us

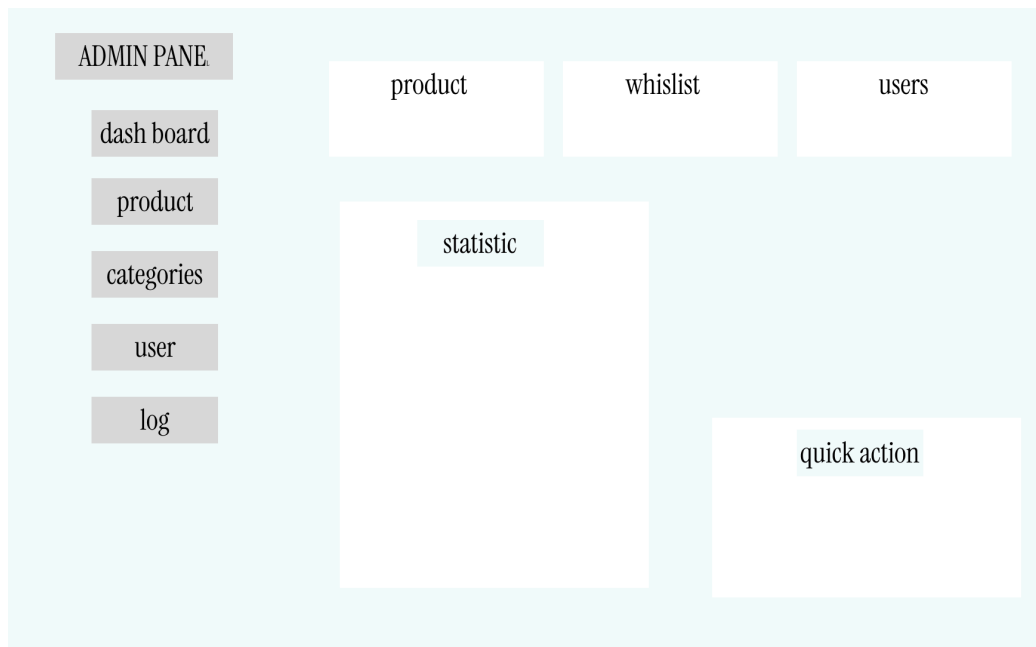
Contact Us

ADD PRODUCT



Insert Picture

ADMIN DASHBOARD



ADMIN PRODUCT CONTROL



ADMIN CATEGORY CONTROL

ADMIN PANE

dash board

product

categories

user

LOG

categories

category name

add categories

name	count	delete
------	-------	--------

ADMIN USER CONTROL

ADMIN PANE

dash board

product

categories

user

LOG

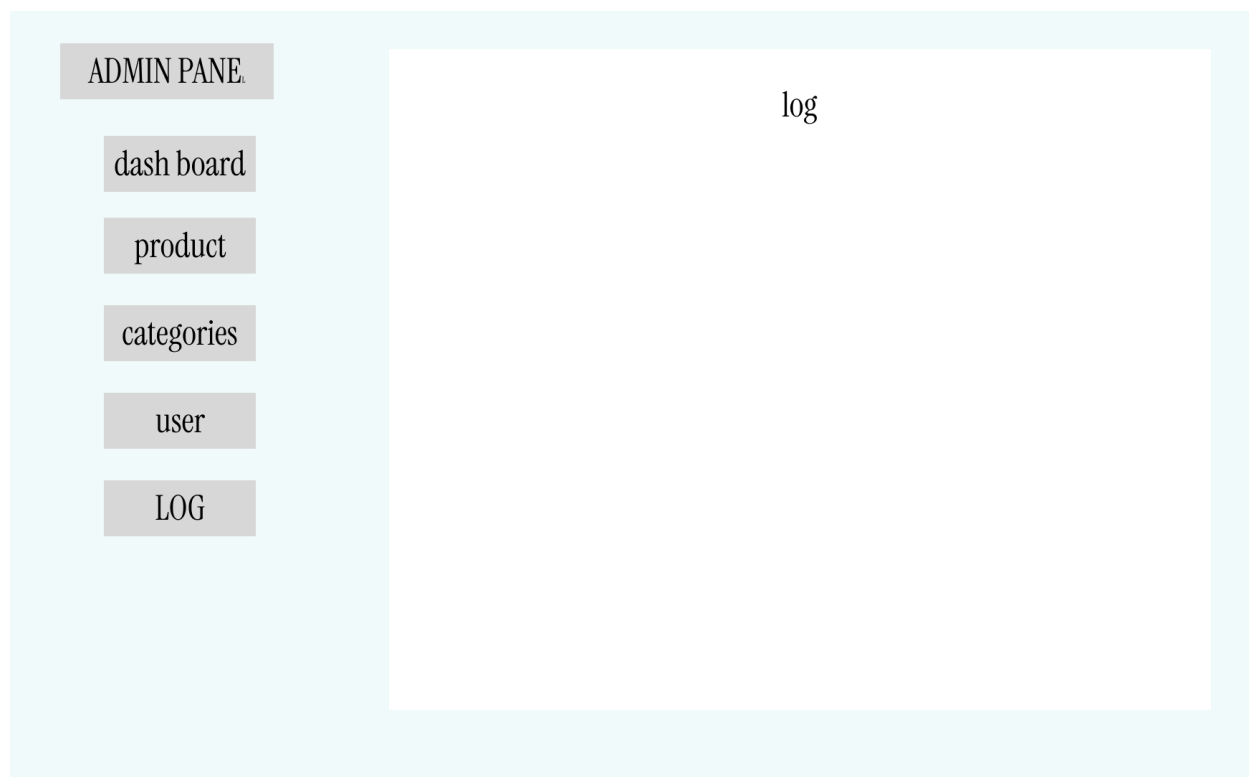
categories

category name

add categories

name	count	delete
------	-------	--------

ADMIN LOG CONTROL



Development Process

Process :

1. Project Tracking

ทีมใช้ github ในการติดตามสถานะของงาน โดยแบ่งงานออกเป็น 3 สถานะหลัก ได้แก่:

1. To Do – งานที่ต้องทำ
2. In Progress – งานที่กำลังดำเนินการ
3. Done – งานที่เสร็จแล้วสมาชิกในทีมจะทำการอัปเดตสถานะของงานทุกครั้งที่มีความคืบหน้า เพื่อให้ทุกคนสามารถติดตามภาพรวมของโปรเจกต์ได้แบบ real-time

Methods :

2. Scrum & Meeting

ทีมใช้แนวคิดของ Agile Scrum ในการพัฒนา โดยมีการประชุมอย่างสม่ำเสมอ:

1. Daily / Every 1 week Scrum
2. อัปเดตว่าแต่ละคนทำอะไรไปแล้ว
3. มีปัญหาอะไรหรือไม่

4. แผนงานถัดไปคืออะไร
5. Sprint Planning วางแผนงานก่อนเริ่มแต่ละช่วง
6. Retrospective สรุปสิ่งที่ทำได้ดี และสิ่งที่ควรปรับปรุง

Tools:

1. Git & GitHub: for version control and collaboration
2. Visual Studio Code – เขียนโค้ด
3. Figma – ออกแบบ UI/UX

Communication:

1. Discord – คุยงานและแชร์ข้อมูล
2. Instagram- สำหรับติดต่อแบบเร่งด่วน

Improvements from Phase 1:

จาก Phase 1 ทีมได้มีการพัฒนาเพิ่มเติมในด้านกระบวนการทำงาน เช่น:
 เพิ่มการใช้เครื่องมือ Project Tracking เพื่อจัดการงานอย่างเป็นระบบ
 มีการกำหนดมาตรฐานในการตั้งชื่อ branch และ commit message
 เพิ่มความถี่ของการประชุม Scrum เพื่อแก้ปัญหาได้รวดเร็วขึ้น
 ใช้เครื่องมือสื่อสารที่ชัดเจน ทำให้การทำงานเป็นทีมมีประสิทธิภาพมากขึ้น

อธิบายการทำงานของ program

1. การทำงานของโปรแกรม (Program Workflow)

โปรแกรมนี้นี้เป็นเว็บแอปสำหรับจัดการและค้นหาสินค้า โดยใช้ Express.js เป็น backend และ EJS เป็น template engine สำหรับแสดงผลหน้าเว็บฝั่ง server จุดเด่นของระบบคือมีการแยกการทำงานออกเป็นหลายส่วน เช่น หน้าแสดงสินค้า, ระบบสมาชิก, ระบบผู้ดูแล, ระบบ log และการจัดการหมวดหมู่สินค้า โดยมีรายละเอียดดังนี้

● HTTP Methods:

- **GET:** ใช้สำหรับดึงข้อมูลสินค้า (Products), รายการหมวดหมู่ (Categories), และหน้าเว็บหลัก (Pages)
- **POST:** ใช้ในการจัดการระบบสมาชิก เช่น การลงชื่อเข้าใช้ (Login) และการสมัครสมาชิก (Registration) รวมถึงการจัดการข้อมูลในส่วน Admin
- **Template Engine:** โปรแกรมใช้ **Template** สำหรับการแสดงผลฝั่ง Client (อ้างอิงจากไฟล์ในโฟลเดอร์ public/js และ views หากมี) เพื่อแยกส่วน Logic ออกจาก User Interface
- **API & Database:** มีการเรียกใช้ **Supabase API** เพื่อจัดการฐานข้อมูลและระบบ Authentication
- **Calculation & Graphs:** มีการใช้ **Morris.js** และ **Raphael.js** สำหรับการวาดกราฟแสดงผลสถิติหรือข้อมูลการวิเคราะห์ (Profiling)

รายละเอียด routes หลัก

Method	Route Path	View / Template	รายละเอียด
GET	/	home.ejs	หน้าแรกของเว็บไซต์ (แสดงสินค้า)
GET	/wishlist	wishlist.ejs	แสดงรายการสินค้าที่บันทึกไว้
GET	/product/:id	product_detail.ejs	รายละเอียดสินค้าแต่ละชิ้น
POST	/product/:id/buy	-	ดำเนินการสั่งซื้อสินค้า (Redirect/Logic)
GET	/about	about.ejs	หน้าเกี่ยวกับเรา

GET	/login	login.ejs	หน้าฟอร์มเข้าสู่ระบบ
GET	/register	register.ejs	หน้าฟอร์มสมัครสมาชิก
GET	/forgot-password	forgot-password.ejs	หน้าขอรหัสผ่านใหม่

2. ระบบยืนยันตัวตน (**routes/auth.js**)

กลุ่มนี้จัดการเกี่ยวกับ Logic เบื้องหลังการจัดการ User

Method	Route Path	การทำงาน
POST	/login	ตรวจสอบรหัสผ่านและสร้าง Session/Token
POST	/register	บันทึกข้อมูลผู้ใช้ใหม่ลง Database
GET	/logout	ล้าง Session และออกจากระบบ

3. ระบบจัดการสินค้า (**routes/products.js**)

สำหรับ Admin หรือระบบหลังบ้านในการจัดการ Stock

Method	Route Path	View / Template	รายละเอียด
GET	/add	product_form.ejs	แสดงฟอร์มเพิ่มสินค้า
GET	/edit/:id	product_form.ejs	แสดงฟอร์มแก้ไขข้อมูลสินค้าเดิม
POST	/add	-	บันทึกสินค้าใหม่ลง Database
POST	/edit/:id	-	อัปเดตข้อมูลสินค้า
POST	/delete/:id	-	ลบสินค้าออกจากระบบ

4. ระบบจัดการหมวดหมู่และแอดมิน (**routes/categories.js & admin.js**)

File	Method	Route Path	View / Template	รายละเอียด
------	--------	------------	-----------------	------------

Categories	POST	/add	-	เพิ่มหมวดหมู่สินค้าใหม่
Categories	POST	/delete/:id	-	ลบหมวดหมู่สินค้า
Admin	GET	/	admin.ejs	หน้า Dashboard สำหรับแอดมิน
Admin	POST	/users/delete/:id	-	ลบผู้ใช้งานออกจากระบบ

โครงสร้าง Partial (Reusable Components)

เป็นส่วนที่ถูกเรียกใช้ซ้ำในหลายๆ หน้าเพื่อลดความซ้ำซ้อนของ Code:

- header.ejs: ส่วนหัวเว็บไซต์, Menu Navigation
- footer.ejs: ส่วนท้ายเว็บไซต์, ข้อมูลติดต่อ, Copyright
- product_card.ejs: การแสดงผลสินค้าแบบการ์ด (ใช้ซ้ำในหน้า Home, Wishlist)

Note: สำหรับหน้า error.ejs มักจะถูกเรียกผ่าน Middleware เมื่อไม่พบเส้นทาง (404) หรือเกิดข้อผิดพลาดภายในระบบ (500) โดยไม่ได้ระบุใน Route หลักโดยตรง

1. การเรียกใช้งาน API และการประมวลผลข้อมูล

1.1 การเรียก API

- Frontend: ไม่มีการเรียกใช้งาน Public API ภายนอกโดยตรงจากหน้าเว็บเบราว์เซอร์
- Backend: ระบบใช้สถาปัตยกรรมแบบ Server-side API Access โดยเรียกใช้งาน Supabase ผ่านไลบรารี **@supabase/supabase-js** ในไฟล์ **utils/supabase.js**

- Data Management: ในฝั่ง Route มีการใช้คำสั่ง **supabase.from(...).select / insert / update / delete** สำหรับจัดการข้อมูลสินค้า หมวดหมู่ และการเก็บบันทึกประวัติ (Log)

1.2 การคำนวณภายในโปรแกรม

- หน้า Home และ Admin: มีการคำนวณมูลค่ารวม (**totalValue**) จากผลรวมราคาสินค้าทั้งหมด, จำนวนจำนวนสินค้าโปรโมชั่น (**promoCount**), จำนวนสินค้าแยกตามแพลตฟอร์ม (เช่น **shopeeCount, lazadaCount**) และจำนวนผู้ใช้งานระบบ (**userCount**)
- หน้า Wishlist: มีระบบกรองข้อมูล (Filter) ตาม **category, platform, search** และระบบเรียงลำดับ (Sort) แบบ **price-asc, price-desc** และ **name-asc**
- หน้า Product Detail: มีการบันทึก Log การทำรายการเมื่อผู้ใช้กดปุ่มสั่งซื้อ (Buy)

5. ตาราง Unit Test Cases

1. Test Suite (ไฟล์ทดสอบ)	2. Describe (เป้าหมาย/ฟังก์ชัน)	3. Test Unit (เงื่อนไข/สถานการณ์)	4. Describe (ผลลัพธ์ที่คาดหวัง)
admin.routes.test.js	GET /admin	เมื่อผู้ใช้ไม่ใช่ Admin	คืนค่า 403 (Forbidden)
	GET /admin	เมื่อผู้ใช้เป็น Admin	แสดงหน้า Dashboard และสถิติ
	POST /admin/users/delete/:id	เมื่อพยายามลบบัญชีของตัวเอง	บล็อกการทำงาน (ห้ามลบตัวเอง)

	POST /admin/users/delete/:id	เมื่อเลือกลบผู้ใช้อื่น	ลบผู้ใช้งานคนอื่นออกจากระบบ สำเร็จ
app.test.js	App Initialization	รัน Environment NODE_ENV=test	ข้ามการเรียกคำสั่ง db.seed()
	App Initialization	รัน Environment NODE_ENV=production	เรียกใช้คำสั่ง db.seed()
auth.middleware.test.js	requireLogin	เมื่อยังไม่ได้เข้าสู่ระบบ	Redirect ผู้ใช้ไปที่หน้า /login
	requireLogin	เมื่อเข้าสู่ระบบแล้ว	เรียกใช้ next() เพื่อทำงานต่อ
	requireAdmin	เมื่อผู้ใช้ไม่ใช่ Admin	คืนค่าข้อผิดพลาด 403 (Forbidden)
	requireAdmin	เมื่อผู้ใช้เป็น Admin	เรียกใช้ next() เพื่อทำงานต่อ
auth.routes.test.js	POST /auth/login	ข้อมูล (Credentials) ไม่ถูกต้อง	แสดงหน้าข้อผิดพลาด

	POST /auth/login	เข้าสู่ระบบสำเร็จ (Admin)	สร้าง Session และ Redirect ไปหน้าหลัก
	POST /auth/register	กรอกข้อมูลไม่ครบถ้วน	แสดงข้อผิดพลาด
	POST /auth/register	Username ซ้ำกับในระบบ	ปฏิเสธการสมัครและแสดงข้อผิดพลาด
	POST /auth/register	Email ซ้ำกับในระบบ	ปฏิเสธการสมัครและแสดงข้อผิดพลาด
	POST /auth/register	ข้อมูลถูกต้องครบถ้วน	สร้างผู้ใช้ใหม่และ Redirect ไปหน้า Wishlist
	POST /auth/register	กระบวนการ Insert ล้มเหลว	แสดงหน้าข้อผิดพลาด
	GET /auth/logout	เรียกใช้งาน Logout	ทำลาย Session และ Redirect ไปหน้า Home
categories.routes.test.js	POST /admin/categories/add	เมื่อผู้ใช้ไม่ใช่ Admin	คืนค่า 403 (Forbidden)

	POST /admin/categories/add	ไม่ได้ส่งชื่อหมวดหมู่ (Name is missing)	Redirect กลับโดยไม่มีการประมวลผล
	POST /admin/categories/add	ข้อมูลถูกต้อง	บันทึกหมวดหมู่ใหม่ลงฐานข้อมูล (ทำครั้งเดียว)
	POST /admin/categories/add	ชื่อหมวดหมู่มีอยู่แล้ว (ซ้ำ)	ข้ามการบันทึกข้อมูล (Skip)
	POST /admin/categories/delete/:id	ส่งลบโดย Admin	ลบหมวดหมู่สำเร็จ
db.utils.test.js	ฟังก์ชัน read()	ทำงานสำเร็จ (Success)	คืนค่ารายการแถวข้อมูล (Rows)
	ฟังก์ชัน read()	เกิดข้อผิดพลาด (Error)	คืนค่า Array ว่าง ([])
	ฟังก์ชัน insert()	ทำงานสำเร็จ (Success)	คืนค่า Success Payload
	ฟังก์ชัน insert()	เกิดข้อผิดพลาด (Error)	คืนค่า Failure Payload

	ฟังก์ชัน update()	เกิดข้อผิดพลาด (Error)	บันทึก Log ข้อผิดพลาด แต่ฟังก์ชันยังทำงานต่อได้
	ฟังก์ชัน remove()	เกิดข้อผิดพลาด (Error)	บันทึก Log ข้อผิดพลาด แต่ฟังก์ชันยังทำงานต่อได้
	ฟังก์ชัน seed()	ตารางผู้ใช้งานเปล่า (Empty table)	บันทึกผู้ใช้ Admin เริ่มต้น (Default)
	ฟังก์ชัน seed()	มีข้อมูลผู้ใช้งานในระบบแล้ว	ไม่ทำงานซ้ำ (Does not insert)
main.test.js	ฟังก์ชันเริ่มต้นของ main.js	เมื่อโหลดสคริปต์เสร็จสิ้น	ลงทะเบียน Event Listener สำหรับ Click และ Submit
	Event: Click Listener	คลิกรอบนอกเมนู (Clicking outside)	ซ่อนเมนู (Hides the menu)
	Event: Submit Listener	การกดลบข้อมูล (Delete)	ป้องกันการลบหากไม่ปลอดภัย (Unsafe delete)

pages.routes.test.js	GET /	โหลดหน้าแรก	แสดงหน้า Home, สถิติ และ สินค้า 4 รายการแรก
	GET /wishlist	ค้นหา กรอง และจัดเรียงข้อมูล	ผลลัพธ์แสดงสินค้าตามเงื่อนไขได้ อย่างถูกต้อง
	GET /product/:id	ค้นหาสินค้าพบ	แสดงหน้ารายละเอียดของสินค้า
	GET /product/:id	ค้นหาสินค้าไม่พบ (Missing)	คืนค่าข้อผิดพลาด 404 (Not Found)
	POST /product/:id/buy	ทำรายการสั่งซื้อสำเร็จ	บันทึก Log การคลิกและคืนค่าลิงก์ ไปยังร้านค้า
	POST /product/:id/buy	สินค้าไม่มีในระบบ	คืนค่าลิงก์สำรอง (Fallback link)
	GET /login & /register	ผู้ใช้เข้าสู่ระบบอยู่แล้ว	Redirect ออกจากหน้าไปหน้าอื่น

	GET /forgot-password	เรียกดูหน้าเว็บ	แสดงผลหน้าฟอร์มได้อย่างถูกต้อง
products.routes.test.js	GET /products/add	ยังไม่ได้เข้าสู่ระบบ	Redirect ไปหน้า Login
	GET /products/add	เข้าสู่ระบบแล้ว	แสดงฟอร์มและรายชื่อหมวดหมู่
	POST /products/add	ข้อมูลจำเป็นไม่ครบ	Redirect กลับไปยังหน้าฟอร์ม
	POST /products/add	ข้อมูลครบถ้วน	บันทึกสินค้า/หมวดหมู่ และ Redirect สำเร็จ
	GET /products/edit/:id	ไม่พบสินค้านั้นในระบบ	คืนค่า 404 (Not Found)
	GET /products/edit/:id	เป็นเจ้าของสินค้า	แสดงแบบฟอร์มเพื่อแก้ไขข้อมูล
	POST /products/edit/:id	ผู้ใช้ไม่มีสิทธิ์ (ไม่ใช่เจ้าของ)	คืนค่า 403 (Forbidden)

	POST /products/edit/:id	ข้อมูลถูกต้องและมีสิทธิ์	อัปเดตข้อมูลและสร้างหมวดหมู่ใหม่(หากยังไม่มี)
	POST /products/delete/:id	ไม่พบสินค้านั้นในระบบ	คืนค่า 404 (Not Found)
	POST /products/delete/:id	ผู้ใช้ไม่มีสิทธิ์	คืนค่า 403 (Forbidden)
	POST /products/delete/:id	สั่งลบโดยเจ้าของสินค้า	ลบข้อมูลสินค้าออกสำเร็จ
supabase.utils.test.js	createClient	ตั้งค่าเริ่มต้น	เรียกใช้ค่า Environment Variables จาก process.env
	Supabase Client Method	เรียกดู Export Module	Client มีเมธอด .from() สำหรับเลือกตาราง
	Data Operation: Select	ส่งคำสั่งดึงข้อมูล	สามารถดึงข้อมูล (Query) จากตารางได้สำเร็จ
	Data Operation: Insert	ส่งคำสั่งเพิ่มข้อมูล	สามารถเพิ่มข้อมูลลงในตารางได้สำเร็จ

	Data Operation: Update	ส่งคำสั่งแก้ไขข้อมูล	สามารถแก้ไขข้อมูลในตารางได้สำเร็จ
	Data Operation: Delete	ส่งคำสั่งลบข้อมูล	สามารถลบข้อมูลในตารางได้สำเร็จ
	Method: .eq()	สั่งกรองข้อมูล	ฟังก์ชันสามารถใช้เป็น Filter หาข้อมูลได้
	Integration: Routing	ดึงข้อมูลผ่าน Route	ไฟล์ Route ใช้งาน Supabase Client ได้อย่างถูกต้อง
	Integration: Wishlist	ดึงข้อมูลใน Route Wishlist	ดึงข้อมูลสินค้าที่ตรงเงื่อนไขได้
	Integration: Product Detail	Render หน้ารายละเอียด	แสดงผลข้อมูล Mock Data ได้ถูกต้องตาม View

3. ตารางทดสอบ Class และ Module อื่นๆ

โปรเจกต์นี้ไม่ได้ใช้โครงสร้างแบบ OOP (Custom Class-based) แต่ใช้สถาปัตยกรรมแบบ Module/Function-based ร่วมกับ Express Router

No.	Class / Module	สถานะ	หมายเหตุ
1	Custom Class	ไม่มี	โปรเจกต์ไม่มีการสร้าง Class ด้วยตนเอง
2	Express Router	มี	ใช้เป็น Module สำหรับจัดการ Routing
3	Utility Function	มี	ฟังก์ชันช่วยเหลือ เช่น read, insert, update, remove, requireLogin, requireAdmin

3. Test Case Code

/app

-Routes หลัก (/login, /register, /about) return HTTP 200

```
// Test that basic routes (/login, /register, /about) return HTTP 200 OK status
test.each([
  ['/login'],
  ['/register'],
  ['/about'],
])('GET %s responds with 200', async (path) => {
  const res = await request(app).get(path);
  expect(res.status).toBe(200);
});
```

-Test env ไม่เรียก db.seed (ป้องกัน data mutation)

```
// Verify that in test environment (NODE_ENV=test), app startup does NOT call db.seed() to avoid data mutation
test('env NODE_ENV=test does not call db.seed', () => {
  process.env.NODE_ENV = 'test';
  require('../app');
  const db = require('../utils/db');
  expect(db.seed).not.toHaveBeenCalled();
});
```

-Production env เรียก db.seed (init default data)

```
// Verify that in production environment (NODE_ENV=production), app startup calls db.seed() to initialize
test('env NODE_ENV=production calls db.seed', () => {
  const db = require('../utils/db');
  db.seed.mockClear();
  process.env.NODE_ENV = 'production';
  jest.resetModules();
  jest.doMock('../utils/db', () => ({
    read: jest.fn().mockResolvedValue([]),
    insert: jest.fn().mockResolvedValue({ success: true }),
    update: jest.fn().mockResolvedValue(),
    remove: jest.fn().mockResolvedValue(),
    seed: db.seed
  }));
  require('../app');
  expect(db.seed).toHaveBeenCalled();
});
```

routes/admin

-หน้า admin กันคนที่ไม่ใช่ admin โดยส่ง status 403

```
// Test that non-admin users get 403 Forbidden when accessing /admin dashboard
test('GET /admin returns 403 when not admin', async () => {
  const res = await request(makeApp({ id: 'u1', role: 'user' })).get('/admin');
  expect(res.status).toBe(403);
});
```

-Admin เห็น stats + logs

```
// Test that admin users can access dashboard and see correct stats (product counts, v
test('GET /admin renders dashboard stats for admins', async () => {
  const res = await request(makeApp({ id: 'admin-1', role: 'admin' })).get('/admin');

  expect(res.status).toBe(200);
  expect(res.body.view).toBe('admin');
  expect(res.body.stats).toEqual(expect.objectContaining({
    total: 3,
    totalValue: 500,
    shopeeCount: 2,
    lazadaCount: 1,
    promoCount: 1,
    userCount: 2
  }));
  expect(res.body.logs).toHaveLength(50);
});
```

-Admin ลบตัวเองไม่ได้

```
// Test that admin cannot delete their own user account (self-protection)
test('POST /admin/users/delete/:id blocks deleting self', async () => {
  const res = await request(makeApp({ id: 'admin-1', role: 'admin' }))
    .post('/admin/users/delete/admin-1');

  expect(db.remove).not.toHaveBeenCalled();
  expect(res.status).toBe(302);
  expect(res.headers.location).toBe('/admin');
});
```

-Admin ลบ user อื่นได้

```
// Test that admin can delete other non-admin users
test('POST /admin/users/delete/:id deletes other users', async () => {
  const res = await request(makeApp({ id: 'admin-1', role: 'admin' }))
    .post('/admin/users/delete/u2');

  expect(db.remove).toHaveBeenCalledWith('users', 'u2');
  expect(res.status).toBe(302);
  expect(res.headers.location).toBe('/admin');
});
});
```

-middleware/auth

-ถ้าไม่ล็อกอิน ให้ไปหน้า login

```
// Test that requireLogin middleware redirects unauthenticated users to login page with return URL
test('requireLogin redirects to /login when not logged in', () => {
  const req = { session: {}, originalUrl: '/products/add' };
  const res = createRes();
  const next = jest.fn();

  requireLogin(req, res, next);

  expect(res.redirect).toHaveBeenCalledWith('/login?redirect=' + encodeURIComponent('/products/add'));
  expect(next).not.toHaveBeenCalled();
});
```

-ถ้าล็อกอินแล้ว ผ่านไปได้เลย

```
// Test that requireLogin middleware allows authenticated users to proceed
test('requireLogin calls next when logged in', () => {
  const req = { session: { user: { id: 'u1' } }, originalUrl: '/wishlist' };
  const res = createRes();
  const next = jest.fn();

  requireLogin(req, res, next);

  expect(res.redirect).not.toHaveBeenCalled();
  expect(next).toHaveBeenCalledTimes(1);
});
```

-ถ้าไม่ใช่ admin ให้ error 403

```
test('requireAdmin returns 403 error when not admin', () => {
  const cases = [
    { session: {} },
    { session: { user: { id: 'u1', role: 'user' } } }
  ];

  for (const req of cases) {
    const res = createRes();
    const next = jest.fn();

    requireAdmin(req, res, next);

    expect(res.status).toHaveBeenCalledWith(403);
    expect(res.render).toHaveBeenCalledWith('error', expect.objectContaining({ message: expect.any(String) }));
    expect(next).not.toHaveBeenCalled();
  }
});
```

-ถ้าเป็น admin ผ่านไปได้เลย

```
// ทดสอบ: requireAdmin เรียก next() เมื่อเป็น admin
test('requireAdmin calls next when admin', () => {
  const req = { session: { user: { id: 'admin', role: 'admin' } } };
  const res = createRes();
  const next = jest.fn();

  requireAdmin(req, res, next);

  expect(res.redirect).not.toHaveBeenCalled();
  expect(next).toHaveBeenCalledTimes(1);
});
```

routes/auth

-Login รหัสผิด → error

```
// Test login failure with wrong password - should re-render login form with error message
test('POST /auth/login renders error on bad credentials', async () => {
  db.read.mockResolvedValue([
    { id: 'u1', username: 'alice', password: 'hashed-password', email: 'a@example.com', role_id: 2 }
  ]);
  bcrypt.compareSync.mockReturnValue(false);

  const res = await request(makeApp())
    .post('/auth/login')
    .send({ username: 'alice', password: 'wrong', redirect: '/wishlist' });

  expect(res.status).toBe(200);
  expect(res.body.view).toBe('login');
  expect(res.body.redirect).toBe('/wishlist');
  expect(res.body.error).toContain('Username');
});
```

-Login รหัสผิด → error

```
// Test successful login for admin user - should set session and redirect to requested page
test('POST /auth/login stores session and redirects for admin users', async () => {
  db.read.mockResolvedValue([
    { id: 'admin-1', username: 'admin', password: 'hashed-password', email: 'admin@example.com', role_id: 1 }
  ]);
  bcrypt.compareSync.mockReturnValue(true);

  const app = makeApp();
  const res = await request(app)
    .post('/auth/login')
    .send({ username: 'admin', password: 'secret', redirect: '/wishlist' });

  expect(res.status).toBe(302);
  expect(res.headers.location).toBe('/wishlist');
});
```

-Register ขาด field

```
// Test register validation - missing required fields should re-render form with Thai error message
test('POST /auth/register renders error when fields are missing', async () => {
  const res = await request(makeApp())
    .post('/auth/register')
    .send({ email: '', username: 'newuser', password: '' });

  expect(res.status).toBe(200);
  expect(res.body.view).toBe('register');
  expect(res.body.error).toContain('กรุณากรอก');
});
```

-Username ซ้ำ

```
// Test register uniqueness validation - duplicate username should fail
test('POST /auth/register rejects duplicate username', async () => {
  db.read.mockResolvedValue([
    { id: 'u1', username: 'taken', email: 'taken@example.com', password: 'pw', role_id: 2 }
  ]);

  const res = await request(makeApp())
    .post('/auth/register')
    .send({ email: 'new@example.com', username: 'taken', password: 'pw' });

  expect(res.status).toBe(200);
  expect(res.body.view).toBe('register');
  expect(res.body.error).toContain('Username');
});
```

-Emailซ้ำ

```
// Test register uniqueness validation - duplicate email should fail
test('POST /auth/register rejects duplicate email', async () => {
  db.read.mockResolvedValue([
    { id: 'u1', username: 'taken', email: 'taken@example.com', password: 'pw', role_id: 2 }
  ]);

  const res = await request(makeApp())
    .post('/auth/register')
    .send({ email: 'taken@example.com', username: 'newuser', password: 'pw' });

  expect(res.status).toBe(200);
  expect(res.body.view).toBe('register');
  expect(res.body.error).toContain('Email');
});
```

-DB insert fail

```
// Test successful user registration - creates hashed password, default user role, and redirects to wishlist
test('POST /auth/register creates user and redirects to wishlist', async () => {
  db.read.mockResolvedValue([]);
  db.insert.mockResolvedValue({ success: true });

  const app = makeApp();
  const res = await request(app)
    .post('/auth/register')
    .send({ email: 'new@example.com', username: 'newuser', password: 'secret' });

  expect(db.insert).toHaveBeenCalledWith('users', [
    expect.objectContaining({
      id: 'new-user-id',
      username: 'newuser',
      email: 'new@example.com',
      password: 'hashed-password',
      role_id: 2
    })
  ]);
  expect(res.status).toBe(302);
  expect(res.headers.location).toBe('/wishlist');
});
```

-Logout ลบ session

```
// Test logout - destroys session and redirects to home page
test('GET /auth/logout destroys session and redirects home', async () => {
  const res = await request(makeApp()).get('/auth/logout');

  expect(res.status).toBe(302);
  expect(res.headers.location).toBe('/');
});
```

routes/categories

-ไม่ใช่อ admin status error ได้ 403

```
// Test admin middleware - non-admin cannot add categories (403)
test('POST /admin/categories/add returns 403 when not admin', async () => {
  const res = await request(makeApp()).post('/admin/categories/add').send({ name: 'new-category' });
  expect(res.status).toBe(403);
});
```

-ชื่อ category ว่าง → redirect

```
// Test category add validation - empty name redirects without DB query
test('POST /admin/categories/add redirects without reading when name is missing', async () => {
  const res = await request(makeApp({ id: 'admin-id', role: 'admin' }))
    .post('/admin/categories/add')
    .send({ name: '' });

  expect(res.status).toBe(302);
  expect(res.headers.location).toBe('/admin');
  expect(db.read).not.toHaveBeenCalled();
});
```

- Add ใหม่

```
// Test admin adds new category - inserts only if not exists
test('POST /admin/categories/add inserts new category once', async () => {
  const res = await request(makeApp({ id: 'admin-id', role: 'admin' }))
    .post('/admin/categories/add')
    .send({ name: 'new-category' });

  expect(db.insert).toHaveBeenCalledWith('categories', expect.objectContaining({
    name: 'new-category'
  }));

  expect(res.status).toBe(302);
  expect(res.headers.location).toBe('/admin');
});
```

- ซ้ำไม่ insert

```
// Test idempotency - duplicate category names are skipped (no insert)
test('POST /admin/categories/add skips duplicate categories', async () => {
  const res = await request(makeApp({ id: 'admin-id', role: 'admin' }))
    .post('/admin/categories/add')
    .send({ name: 'tech' });

  expect(db.insert).not.toHaveBeenCalled();
  expect(res.status).toBe(302);
  expect(res.headers.location).toBe('/admin');
});
```

-ลบ categories

```
// Test admin category deletion - removes specified category ID
test('POST /admin/categories/delete/:id removes a category for admins', async () => {
  const res = await request(makeApp({ id: 'admin-id', role: 'admin' }))
    .post('/admin/categories/delete/c1');

  expect(db.remove).toHaveBeenCalledWith('categories', 'c1');
  expect(res.status).toBe(302);
  expect(res.headers.location).toBe('/admin');
});
});
```

utils/db

-Supabase select() สำเร็จ return array data

```
// Test db.read success path - returns data array when Supabase select succeeds
test('read returns rows on success', async () => {
  const { db } = await loadDbWithBehavior({
    selectResult: { data: [{ id: 'p1' }], error: null },
    insertSelectResult: { data: [], error: null },
    eqResult: { error: null }
  });

  await expect(db.read('products')).resolves.toEqual([{ id: 'p1' }]);
});
```

- Supabase select error → return [] + console.error('READ ERROR')

```
// Test db.read error handling - returns empty array and logs error when Supabase fails
test('read returns an empty array on error', async () => {
  const { db } = await loadDbWithBehavior({
    selectResult: { data: null, error: new Error('read failed') },
    insertSelectResult: { data: [], error: null },
    eqResult: { error: null }
  });

  await expect(db.read('products')).resolves.toEqual([]);
  expect(console.error).toHaveBeenCalledWith('READ ERROR:', expect.any(Error));
});
```

- ถ้า insert success ให้ส่งข้อมูลกลับ

```
// Test db.insert success - returns {success: true, data} when Supabase insert succeeds
test('insert returns success payload on success', async () => {
  const { db } = await loadDbWithBehavior({
    selectResult: { data: [], error: null },
    insertSelectResult: { data: [{ id: 'x1' }], error: null },
    eqResult: { error: null }
  });

  await expect(db.insert('products', [{ id: 'x1' }])).resolves.toEqual({
    success: true,
    data: [{ id: 'x1' }]
  });
});
```

- insert database ถ้า error ให้ส่งข้อมูลกลับมา

```
// Test db.insert error handling - returns {success: false, error}
test('insert returns failure payload on error', async () => {
  const { db } = await loadDbWithBehavior({
    selectResult: { data: [], error: null },
    insertSelectResult: { data: null, error: new Error('insert failed') },
    eqResult: { error: null }
  });

  await expect(db.insert('products', [{ id: 'x1' }])).resolves.toEqual({
    success: false,
    error: expect.any(Error)
  });
  expect(consoleError).toHaveBeenCalledWith('INSERT ERROR:', expect.any(Error));
});

test('update logs errors but still resolves', async () => {
  const { db } = await loadDbWithBehavior({
    selectResult: { data: [], error: null },
    insertSelectResult: { data: [], error: null },
    eqResult: { error: new Error('update failed') }
  });

  await expect(db.update('products', 'p1', { name: 'Updated' })).resolves.toBeUndefined();
  expect(consoleError).toHaveBeenCalledWith('UPDATE ERROR:', expect.any(Error));
});

test('remove logs errors but still resolves', async () => {
  const { db } = await loadDbWithBehavior({
    selectResult: { data: [], error: null },
    insertSelectResult: { data: [], error: null },
    eqResult: { error: new Error('delete failed') }
  });

  await expect(db.remove('products', 'p1')).resolves.toBeUndefined();
  expect(consoleError).toHaveBeenCalledWith('DELETE ERROR:', expect.any(Error));
});
```

-seed empty users ๖๗ insert admin

```
test('seed inserts the default admin user when table is empty', async () => {
  const { supabaseMock } = await loadDbWithBehavior({
    selectResult: { data: [], error: null },
    insertSelectResult: { data: [{ id: 'admin' }], error: null },
    eqResult: { error: null }
  });

  await new Promise((resolve) => setImmediate(resolve));

  expect(supabaseMock.from).toHaveBeenCalledWith('users');
  expect(supabaseMock.tables.users.insert).toHaveBeenCalled();
});
```

-seed มี user ให้ skip

```
test('seed does not insert when users already exist', async () => {
  const { supabaseMock } = await loadDbWithBehavior({
    selectResult: { data: [{ id: 'u1' }], error: null },
    insertSelectResult: { data: [], error: null },
    eqResult: { error: null }
  });

  await new Promise((resolve) => setImmediate(resolve));

  expect(supabaseMock.from).toHaveBeenCalledWith('users');
  expect(supabaseMock.tables.users.insert).not.toHaveBeenCalled();
});
```

routes/page

- wishlist รองรับ category & platform

- Get หน้าสินค้ารายละเอียด

```
// Test wishlist page with full filtering (category, platform, search, sort)
test('GET /wishlist filters, searches, and sorts products', async () => {
  const res = await request(makeApp()).get('/wishlist?category=tech&platform=shopee&search=phone&sort=price-desc');

  expect(res.status).toBe(200);
  expect(res.body.view).toBe('wishlist');
  expect(res.body.products).toHaveLength(1);
  expect(res.body.products[0].id).toBe('p1');
});
```

- test Wishlist filter(Category/platform/search/sort)

```
// Test wishlist page with full filtering (category, platform, search, sort)
test('GET /wishlist filters, searches, and sorts products', async () => {
  const res = await request(makeApp()).get('/wishlist?category=tech&platform=shopee&search=phone&sort=price-desc');

  expect(res.status).toBe(200);
  expect(res.body.view).toBe('wishlist');
  expect(res.body.products).toHaveLength(1);
  expect(res.body.products[0].id).toBe('p1');
});
```

- Product detail

```
// Test product detail page for existing product
test('GET /product/:id renders product detail when found', async () => {
  const res = await request(makeApp()).get('/product/p1');

  expect(res.status).toBe(200);
  expect(res.body.view).toBe('product_detail');
  expect(res.body.product.id).toBe('p1');
});
```

- Product 404 (error page)

```
// Test product detail 404 for non-existent ID
test('GET /product/:id returns 404 when missing', async () => {
  const res = await request(makeApp()).get('/product/missing');

  expect(res.status).toBe(404);
  expect(res.body.view).toBe('error');
});
```

- กด buy แล้วให้ไปที่ link

```
// Test "buy" button - logs user click to logs table and returns affiliate link
test('POST /product/:id/buy logs a click and returns link', async () => {
  const res = await request(makeApp({ id: 'u1', username: 'alice' })).post('/product/p1/buy');

  expect(db.insert).toHaveBeenCalledWith('logs', expect.objectContaining({
    productId: 'p1',
    productName: 'Phone Pro',
    user: 'alice'
  }));
  expect(res.status).toBe(200);
  expect(res.body).toEqual(expect.objectContaining({ ok: true, link: 'https://example.com/1' }));
});
```

- Fallback Link

```
test('POST /product/:id/buy returns fallback link when product is missing', async () => {
  db.read.mockResolvedValueOnce([]);

  const res = await request(makeApp()).post('/product/missing/buy');

  expect(db.insert).not.toHaveBeenCalled();
  expect(res.body.link).toBe('#');
});
```

- Login/register logged

```
// Test auth guard - logged in users redirected from login/register pages
test('GET /login and /register redirect when already signed in', async () => {
  const app = makeApp({ id: 'u1', role: 'user' });
  const loginRes = await request(app).get('/login');
  const registerRes = await request(app).get('/register');
  expect(loginRes.status).toBe(302);
  expect(registerRes.status).toBe(302);
});
```

-Forgot-password

```
// Test forgot password page - public route always accessible
test('GET /forgot-password renders the page', async () => {
  const res = await request(makeApp()).get('/forgot-password');

  expect(res.status).toBe(200);
  expect(res.body.view).toBe('forgot-password');
});
});
```

routes/products

- เพิ่มสินค้าต้องล็อกอิน

```
// ทดสอบ: เพิ่มสินค้า ต้องล็อกอินก่อน ไม่ถึง redirect ไป login
test('GET /products/add redirects when not logged in', async () => {
  const app = makeApp(null);
  const res = await request(app).get('/products/add');
  expect(res.status).toBe(302);
  expect(res.headers.location).toContain('/login');
});
```

- ทดสอบ GET /products/add สำหรับ user ที่ login แล้ว - render form พร้อม categories dropdown

```
// ทดสอบ GET /products/add สำหรับ user ที่ login แล้ว - render form พร้อม categories dropdown
test('GET /products/add renders form with categories when logged in', async () => {
  state.categoriesSelect = { data: [{ id: 'c1', name: 'tech' }], error: null };

  const res = await request(makeApp({ id: 'u1', role: 'user' })).get('/products/add');

  expect(res.status).toBe(200);
  expect(res.body.view).toBe('product_form');
  expect(res.body.mode).toBe('add');
  expect(res.body.categories).toHaveLength(1);
});
```

- ต้องกรอก name และ price ว่าง → redirect ?error=missing

```
// ทดสอบ POST /products/add validation - ฟิลด์ name/price ว่าง → redirect ?error=missing
test('POST /products/add redirects when required fields are missing', async () => {
  const res = await request(makeApp({ id: 'u1', role: 'user' }))
    .post('/products/add')
    .send({ name: '', price: '' });

  expect(res.status).toBe(302);
  expect(res.headers.location).toBe('/wishlist?error=missing');
});
```

- ทดสอบ POST /products/add สำเร็จ - insert product + สร้าง category ถ้าไม่มี + redirect ไป product page

```
// ทดสอบ POST /products/add สำเร็จ - insert product + สร้าง category ถ้าไม่มี + redirect ไป product page
test('POST /products/add inserts product and category then redirects', async () => {
  state.categoriesSelect = { data: [], error: null };

  const res = await request(makeApp({ id: 'u1', role: 'user' }))
    .post('/products/add')
    .send({
      name: 'New Gadget',
      price: '199',
      platform: 'shopee',
      category: 'tech',
      link: 'https://example.com/product',
      description: 'A useful gadget',
      imgUrl: 'https://example.com/image.png',
      isPromo: 'on'
    });

  expect(supabase.from).toHaveBeenCalledWith('products');
  expect(mockProductsTable.insert).toHaveBeenCalledWith([
    expect.objectContaining({
      id: 'product-uuid',
      name: 'New Gadget',
      price: 199,
      platform: 'shopee',
      category: 'tech',
      isPromo: true,
      createdBy: 'u1'
    })
  ]);
  expect(mockCategoriesTable.insert).toHaveBeenCalled();
  expect(res.status).toBe(302);
  expect(res.headers.location).toBe('/product/product-uuid');
});
```

- ทดสอบ GET /products/edit/:id - product ไม่พบ → 404 error

```
// ทดสอบ GET /products/edit/:id - product ไม่พบ → 404 error
test('GET /products/edit/:id returns 404 when product is missing', async () => {
  state.productsSingle = { data: null, error: null };

  const res = await request(makeApp({ id: 'u1', role: 'user' })).get('/products/edit/missing');

  expect(res.status).toBe(404);
  expect(res.body.view).toBe('error');
});

test('GET /products/edit/:id renders the form for the owner', async () => {
  state.productsSingle = { data: { id: 'p1', createdBy: 'u1', name: 'Phone', price: 100 }, error: null };
  state.categoriesSelect = { data: [{ id: 'c1', name: 'tech' }], error: null };

  const res = await request(makeApp({ id: 'u1', role: 'user' })).get('/products/edit/p1');

  expect(res.status).toBe(200);
  expect(res.body.view).toBe('product_form');
  expect(res.body.mode).toBe('edit');
  expect(res.body.product.id).toBe('p1');
});
```

-ทดสอบ POST /products/delete/:id - user ไม่ใช่ owner → 403 permission denied

```
// ทดสอบ POST /products/delete/:id - user ไม่ใช่ owner → 403 permission denied
test('POST /products/delete/:id returns 403 when user lacks permission', async () => {
  state.productsSingle = { data: { id: 'p1', createdBy: 'owner', name: 'Phone', price: 100 }, error: null };

  const res = await request(makeApp({ id: 'u1', role: 'user' })).post('/products/delete/p1');

  expect(res.status).toBe(403);
  expect(res.body.view).toBe('error');
});
```

-ทดสอบ POST /products/delete/:id สำเร็จ - owner ลบได้ →
mockProductsTable.delete() + redirect /wishlist

```
test('POST /products/delete/:id removes product for the owner', async () => {
  state.productsSingle = { data: { id: 'p1', createdBy: 'u1', name: 'Phone', price: 100 }, error: null };

  const res = await request(makeApp({ id: 'u1', role: 'user' })).post('/products/delete/p1');

  expect(mockProductsTable.delete).toHaveBeenCalled();
  expect(res.status).toBe(302);
  expect(res.headers.location).toBe('/wishlist');
});
```

- ต่อ Supabase ด้วย env ที่ตั้งไว้

```
// ทดสอบ: createClient ถูกเรียกด้วย environment variables
test('createClient is called with env variables from process.env', () => {
  expect(mockCreateClient).toHaveBeenCalled();
});
```

- client พร้อมใช้งาน

```
// ทดสอบ: supabase client ถูก export และมี method from
test('supabase client is exported and has from method', () => {
  expect(supabase).toBeDefined();
  expect(typeof supabase.from).toBe('function');
});
```

- ค้นหาข้อมูล (select)

```
// ทดสอบ: supabase client สามารถ query ข้อมูลจาก table ได้
test('supabase client can query a table', async () => {
  const result = await supabase.from('products').select();
  expect(result).toBeDefined();
  expect(result.data).toEqual([]);
  expect(result.error).toBeNull();
});
```

- เพิ่มข้อมูล (insert)

```
// ทดสอบ: supabase client สามารถ insert ข้อมูลลง table ได้
test('supabase client can insert into a table', async () => {
  const result = await supabase.from('products').insert([{ name: 'Test Product' }]);
  expect(result).toBeDefined();
});
```

- แก้ไขข้อมูล (update)

```
// ทดสอบ: supabase client สามารถ update ข้อมูลใน table ได้
test('supabase client can update a record', async () => {
  const result = await supabase.from('products').update({ name: 'Updated' });
  expect(result).toBeDefined();
  expect(result.error).toBeNull();
});
```

- ลบข้อมูล (delete)

```
// ทดสอบ: supabase client สามารถ delete ข้อมูลใน table ได้
test('supabase client can delete a record', async () => {
  const result = await supabase.from('products').delete();
  expect(result).toBeDefined();
  expect(result.error).toBeNull();
});
```

- กรองข้อมูลด้วย filter

```
// ทดสอบ: supabase client สามารถ filter ข้อมูลด้วย eq ได้
test('eq method can be used for filtering', async () => {
  const result = await supabase.from('products').eq('id', 1);
  expect(result).toBeDefined();
  expect(result.error).toBeNull();
});
```

-ทดสอบ routes (/home) ใช้ supabase client สำหรับ query

```

test('routes use supabase client for queries', async () => {
  const app = makeApp();
  const res = await request(app).get('/');
  expect(res.status).toBe(200);
});

test('wishlist route queries products', async () => {
  const app = makeApp();
  const res = await request(app).get('/wishlist');
  expect(res.status).toBe(200);
});

test('product detail renders view with mock data', async () => {
  const app = makeApp();
  const res = await request(app).get('/product/123');
  expect(res.status).toBe(200);
});
});

```

4. Test Coverage Report

ผลการ test ที่ได้จากการ report:

All files

93.3% Statements 223/239 76.59% Branches 308/341 89.36% Functions 42/47 95.79% Lines 285/294

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
middleware	100%	7/7	100%	6/6
routes	92.15%	188/204	75.2%	94/125
utils	100%	28/28	80%	8/10

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	93.3	76.59	89.36	95.79	
middleware	100	100	100	100	
auth.js	100	100	100	100	
routes	92.15	75.2	87.5	94.97	
admin.js	100	75	100	100	13
auth.js	100	86.95	100	100	17-28
categories.js	100	100	100	100	
pages.js	91.8	85.29	83.33	100	9,38-42,44,77
products.js	82.53	63.33	71.42	85.24	32-33,61,90-91,133-134,157,175
utils	100	80	100	100	
db.js	100	80	100	100	40-52
supabase.js	100	100	100	100	
Test Suites: 10 passed, 10 total					
Tests: 66 passed, 66 total					

-middleware/auth.js

All files / middleware auth.js

100% Statements 7/7

100% Branches 6/6

100% Functions 2/2

100% Lines 7/7

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```
1 function requireLogin(req, res, next) {
2   13x   if (!req.session.user) {
3     2x     return res.redirect('/login?redirect=' + encodeURIComponent(req.originalUrl));
4   }
5   11x   next();
6 }
7
8 function requireAdmin(req, res, next) {
9   3x   if (!req.session.user || req.session.user.role !== 'admin') {
10  2x     return res.status(403).render('error', { message: 'ไม่มีสิทธิ์เข้าถึงหน้านี้' });
11   }
12   1x   next();
13 }
14
15  4x module.exports = { requireLogin, requireAdmin };
16
```

All files / routes admin.js

100% Statements 25/25 75% Branches 3/4 100% Functions 6/6 100% Lines 21/21

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```

1 3x const express = require('express');
2 3x const router = express.Router();
3 3x const { requireAdmin } = require('../middleware/auth');
4 3x const db = require('../utils/db'); // << ใช้ db ตัวใหม่ (ที่มี read/insert/etc)
5
6 // GET Admin dashboard
7 3x router.get('/', requireAdmin, async (req, res) => {
8 1x   const products = await db.read('products');
9 1x   const users = await db.read('users');
10 1x   const categories = await db.read('categories');
11 1x   const logs = await db.read('logs');
12
13 3x   const totalValue = products.reduce((sum, p) => sum + Number(p.price || 0), 0);
14 3x   const shopeeCount = products.filter(p => p.platform === 'shopee').length;
15 3x   const lazadaCount = products.filter(p => p.platform === 'lazada').length;
16 3x   const promoCount = products.filter(p => p.isPromo).length;
17
18 1x   res.render('admin', {
19     products,
20     users,
21     categories,
22     logs: logs.slice(0, 50),
23     stats: {
24       total: products.length,
25       totalValue,
26       shopeeCount,
27       lazadaCount,
28       promoCount,
29       userCount: users.length
30     }
31   });
32 });
33
34
35 // POST Delete user (admin only)
36 3x router.post('/users/delete/:id', requireAdmin, async (req, res) => {
37 2x   const userId = req.params.id;
38
39   // ! กันลมตัวเอง
40 2x   if (userId === req.session.user.id) {
41 1x     return res.redirect('/admin');
42   }
43
44 1x   await db.remove('users', userId);
45
46 1x   res.redirect('/admin');
47 });
48
49 3x module.exports = router;
50

```

All files / routes auth.js

100% Statements 35/35

86.95% Branches 20/23

100% Functions 6/6

100% Lines 32/32

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```

1 3x const express = require('express');
2 3x const router = express.Router();
3 3x const bcrypt = require('bcryptjs');
4 3x const { v4: uuidv4 } = require('uuid');
5 3x const db = require('../utils/db');
6
7 // POST Login
8 3x router.post('/login', async (req, res) => {
9 2x   const { username, password, redirect } = req.body;
10
11 2x   const users = await db.read('users');
12 2x   const user = users.find(u => u.username === username);
13
14 2x   if (!user || !bcrypt.compareSync(password, user.password)) {
15 1x     return res.render('login', {
16       error: 'Username หรือ Password ไม่ถูกต้อง',
17       redirect: redirect || '/wishlist'
18     });
19   }
20
21 1x   req.session.user = {
22     id: user.id,
23     username: user.username,
24     email: user.email,
25     role: user.role_id === 1 ? 'admin' : 'user'
26   };
27
28 1x   res.redirect(redirect || '/wishlist');
29   });
30
31 // POST Register
32 3x router.post('/register', async (req, res) => {
33 5x   const { email, username, password } = req.body;
34
35 5x   if (!email || !username || !password) {
36 1x     return res.render('register', { error: 'กรุณากรอกข้อมูลให้ครบ' });
37   }

```

```

38
39 4x   const users = await db.read('users');
40
41 4x   if (users.find(u => u.username === username)) {
42 1x     return res.render('register', { error: 'Username นี้ถูกใช้งานแล้ว' });
43   }
44
45 3x   if (users.find(u => u.email === email)) {
46 1x     return res.render('register', { error: 'Email นี้ถูกใช้งานแล้ว' });
47   }
48
49 2x   const newUser = {
50     id: uuidv4(),
51     username: username.trim(),
52     email: email.trim(),
53     password: bcrypt.hashSync(password, 10),
54
55     role_id: 2, // ✅ ใช้แทน role
56     created_at: new Date().toISOString() // ✅ snake_case
57   };
58
59 2x   const result = await db.insert('users', [newUser]); // ✅ ต้องเป็น array
60
61 2x   if (!result || result.success === false) {
62 1x     return res.render('register', { error: 'สมัครไม่สำเร็จ' });
63   }
64
65 1x   req.session.user = {
66     id: newUser.id,
67     username: newUser.username,
68     email: newUser.email,
69     role: 'user'
70   };
71
72 1x   res.redirect('/wishlist');
73   });
74
75   // Logout
76 3x   router.get('/logout', (req, res) => {
77 1x     req.session.destroy();
78 1x     res.redirect('/');
79   });
80
81 3x   module.exports = router;

```

All files / routes categories.js

100% Statements 20/20

100% Branches 4/4

100% Functions 3/3

100% Lines 18/18

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```

1 3x const express = require('express');
2 3x const router = express.Router();
3 3x const { v4: uuidv4 } = require('uuid');
4 3x const { requireAdmin } = require('../middleware/auth');
5 3x const db = require('../utils/db'); // ✅ ใช้ db ไม่ใช่ supabase ตรง ๆ
6
7 // POST Add category (admin only)
8 3x router.post('/add', requireAdmin, async (req, res) => {
9 3x   const { name } = req.body;
10
11 3x   if (!name) return res.redirect('/admin');
12
13 2x   const categories = await db.read('categories');
14
15   // เช็คซ้ำไหม
16 3x   const exists = categories.find(c => c.name === name.trim());
17
18 2x   if (!exists) {
19 1x     await db.insert('categories', {
20       id: uuidv4(),
21       name: name.trim()
22     });
23   }
24
25 2x   res.redirect('/admin');
26 });
27
28
29 // POST Delete category (admin only)
30 3x router.post('/delete/:id', requireAdmin, async (req, res) => {
31 1x   const categoryId = req.params.id;
32
33 1x   await db.remove('categories', categoryId);
34
35 1x   res.redirect('/admin');
36 });
37
38 3x module.exports = router;
39

```

All files / routes pages.js

91.8% Statements 56/61 85.29% Branches 29/34 83.33% Functions 15/18 100% Lines 47/47

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```

1 4x const express = require('express');
2 4x const router = express.Router();
3 4x const db = require('../utils/db'); // ✓ 1x db
4
5 // Home
6 4x router.get('/', async (req, res) => {
7 2x   const products = await db.read('products');
8
9 5x   const totalValue = products.reduce((sum, p) => sum + Number(p.price || 0), 0);
10 5x   const promoCount = products.filter(p => p.isPromo).length;
11
12 2x   res.render('home', {
13     products: products.slice(0, 4),
14     stats: { total: products.length, totalValue, promoCount }
15   });
16 });
17
18
19 // Wishlist / Products listing
20 4x router.get('/wishlist', async (req, res) => {
21 2x   let products = await db.read('products');
22 2x   const categories = await db.read('categories');
23
24 2x   const { search, platform, sort, category } = req.query;
25
26 2x   if (category && category !== 'all') {
27 5x     products = products.filter(p => p.category === category);
28   }
29
30 2x   if (platform) {
31 3x     products = products.filter(p => p.platform === platform);
32   }
33
34 2x   if (search) {
35 1x     const q = search.toLowerCase();
36 1x     products = products.filter(p =>
37 2x       p.name.toLowerCase().includes(q) ||
38       (p.description || '').toLowerCase().includes(q)
39     );
40   }
41
42 2x   if (sort === 'price-asc') products.sort((a, b) => a.price - b.price);
43 2x   if (sort === 'price-desc') products.sort((a, b) => b.price - a.price);
44 2x   if (sort === 'name-asc') products.sort((a, b) => a.name.localeCompare(b.name, 'th'));
45
46 2x   res.render('wishlist', {
47     products,
48     categories,
49     query: req.query
50   });
51 });
52

```

```

53
54 // Product detail
55 4x router.get('/product/:id', async (req, res) => {
56 3x   const products = await db.read('products');
57 6x   const product = products.find(p => p.id === req.params.id);
58
59 3x   if (!product) {
60 2x     return res.status(404).render('error', { message: 'ไม่พบสินค้านี้' });
61   }
62
63 1x   res.render('product_detail', { product });
64   });
65
66
67 // Log buy click
68 4x router.post('/product/:id/buy', async (req, res) => {
69 2x   const products = await db.read('products');
70 2x   const product = products.find(p => p.id === req.params.id);
71
72 2x   if (product) {
73 1x     await db.insert('logs', {
74       id: Date.now().toString(),
75       productId: product.id,
76       productName: product.name,
77       user: req.session.user ? req.session.user.username : 'guest',
78       time: new Date().toISOString()
79     });
80   }
81
82 2x   res.json({ ok: true, link: product ? product.link : '#' });
83   });
84
85
86 // About
87 4x router.get('/about', (req, res) => {
88 1x   res.render('about');
89   });
90
91
92 // Login / Register pages
93 4x router.get('/login', (req, res) => {
94 2x   if (req.session.user) return res.redirect('/wishlist');
95 1x   res.render('login', { error: null, redirect: req.query.redirect || '/wishlist' });
96   });
97
98 4x router.get('/register', (req, res) => {
99 2x   if (req.session.user) return res.redirect('/wishlist');
100 1x   res.render('register', { error: null });
101   });
102
103 4x router.get('/forgot-password', (req, res) => {
104 1x   res.render('forgot-password', { sent: false });
105   });
106
107 4x module.exports = router;
108

```

All files / routes products.js

82.53% Statements 52/63

63.33% Branches 38/60

71.42% Functions 5/7

85.24% Lines 52/61

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```

1 3x const express = require('express');
2 3x const router = express.Router();
3 3x const { v4: uuidv4 } = require('uuid');
4 3x const { requireLogin } = require('../middleware/auth');
5 3x const supabase = require('../utils/supabase');
6
7 // POST Add product
8 3x router.post('/add', requireLogin, async (req, res) => {
9 2x   const { name, price, platform, category, link, description, imgUrl, isPromo } = req.body;
10
11 2x   if (!name || !price) {
12 1x     return res.redirect('/wishlist?error=missing');
13   }
14
15 1x   const newProduct = {
16     id: uuidv4(),
17     name: name.trim(),
18     price: Number(price),
19     platform: platform || 'other',
20     category: category ? category.trim() : '',
21     link: link ? link.trim() : '',
22     description: description ? description.trim() : '',
23     imgUrl: imgUrl ? imgUrl.trim() : '',
24     isPromo: isPromo === 'on',
25     createdBy: req.session.user.id,
26     createdAt: new Date().toISOString()
27   };
28
29 // 📌 insert product
30 1x const { error } = await supabase.from('products').insert([newProduct]);
31 1x if (error) {
32   console.error(error);
33   return res.status(500).send('Error adding product');
34 }
35
36 // 📌 add category if not exists
37 1x if (newProduct.category) {
38 1x   const { data: categories } = await supabase.from('categories').select('*');
39
40 1x   if (!categories.find(c => c.name === newProduct.category)) {
41 1x     await supabase.from('categories').insert([
42       { id: uuidv4(), name: newProduct.category }
43     ]);
44   }
45 }
46
47 1x res.redirect('/product/' + newProduct.id);
48 });
49
50 // POST Edit product
51 3x router.post('/edit/:id', requireLogin, async (req, res) => {
52 2x   const { name, price, platform, category, link, description, imgUrl, isPromo } = req.body;
53
54 2x   const { data: products } = await supabase
55     .from('products')
56     .select('*')
57     .eq('id', req.params.id)
58     .single();
59

```

```

60 2x I if (!products) {
61     return res.status(404).render('error', { message: 'ไม่พบสินค้า' });
62 }
63
64 // check permission
65 2x if (
66     products.createdBy !== req.session.user.id &&
67     req.session.user.role !== 'admin'
68 ) {
69 1x     return res.status(403).render('error', { message: 'ไม่มีสิทธิ์แก้ไขสินค้านี้' });
70 }
71
72 1x const updatedProduct = {
73     name: name.trim(),
74     price: Number(price),
75     platform: platform || 'other',
76     category: category ? category.trim() : '',
77     link: link ? link.trim() : '',
78     description: description ? description.trim() : '',
79     imgUrl: imgUrl ? imgUrl.trim() : '',
80     isPromo: isPromo === 'on',
81     updatedAt: new Date().toISOString()
82 };
83
84 1x const { error } = await supabase
85     .from('products')
86     .update(updatedProduct)
87     .eq('id', req.params.id);
88
89 1x I if (error) {
90     console.error(error);
91     return res.status(500).send('Error updating product');
92 }
93
94 // add category if new
95 1x E if (updatedProduct.category) {
96 1x     const { data: categories } = await supabase.from('categories').select('*');
97
98 1x     E if (!categories.find(c => c.name === updatedProduct.category)) {
99 1x         await supabase.from('categories').insert([
100             { id: uuidv4(), name: updatedProduct.category }
101         ]);
102     }
103 }
104
105 1x res.redirect('/product/' + req.params.id);
106 });
107
108 // POST Delete product
109 3x router.post('/delete/:id', requireLogin, async (req, res) => {
110 3x     const { data: product } = await supabase
111         .from('products')
112         .select('*')
113         .eq('id', req.params.id)
114         .single();
115
116 3x if (!product) {
117 1x     return res.status(404).render('error', { message: 'ไม่พบสินค้า' });
118 }
119

```

```

119
120 2x if (
121     product.createdBy !== req.session.user.id &&
122     req.session.user.role !== 'admin'
123 ) {
124 1x   return res.status(403).render('error', { message: 'ไม่มีสิทธิ์ลบสินค้านี้' });
125 }
126
127 1x const { error } = await supabase
128     .from('products')
129     .delete()
130     .eq('id', req.params.id);
131
132 1x I if (error) {
133     console.error(error);
134     return res.status(500).send('Error deleting product');
135 }
136
137 1x res.redirect('/wishlist');
138 });
139
140 // GET Edit form
141 3x router.get('/edit/:id', requireLogin, async (req, res) => {
142 2x   const { data: product } = await supabase
143       .from('products')
144       .select('*')
145       .eq('id', req.params.id)
146       .single();
147
148 2x   if (!product) {
149 1x     return res.status(404).render('error', { message: 'ไม่พบสินค้า' });
150   }
151
152 1x   const { data: categories, error } = await supabase
153       .from('categories')
154       .select('*');
155
156 1x I if (error) {
157     console.error(error);
158   }
159
160 1x   res.render('product_form', {
161       product,
162       categories: categories || [], // ✅ แก่ตรงนี้
163       mode: 'edit'
164   });
165 });
166
167
168 // GET Add form
169 3x router.get('/add', requireLogin, async (req, res) => {
170 1x   const { data: categories, error } = await supabase
171       .from('categories')
172       .select('*');
173
174 1x I if (error) {
175     console.error(error);
176   }
177
178 1x   res.render('product_form', {
179       product: null,
180       categories: categories || [], // ✅ แก่ตรงนี้
181       mode: 'add'
182   });
183 });
184
185 3x module.exports = router;

```

All files / utils db.js

100% Statements 25/25

80% Branches 8/10

100% Functions 5/5

100% Lines 25/25

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```

1  8x  const supabase = require('./supabase');
2
3      // READ
4  async function read(table) {
5  10x    const { data, error } = await supabase
6          .from(table)
7          .select('*');
8
9  10x    if (error) {
10  2x      console.error('READ ERROR:', error);
11  2x      return [];
12    }
13
14  8x    return data;
15  }
16
17      // INSERT
18  async function insert(table, data) {
19  8x    const { data: result, error } = await supabase
20          .from(table)
21          .insert(data)
22          .select();
23
24  8x    if (error) {
25  2x      console.error('INSERT ERROR:', error);
26  2x      return { success: false, error };
27    }
28
29  6x    return { success: true, data: result };
30  }
31
32
33      // UPDATE
34  async function update(table, id, newData) {
35  1x    const { error } = await supabase
36          .from(table)
37          .update(newData)
38          .eq('id', id);
39
40  1x    E if (error) {
41  1x      console.error('UPDATE ERROR:', error);
42    }
43  }
44

```

```

45 // DELETE
46 async function remove(table, id) {
47   1x   const { error } = await supabase
48         .from(table)
49         .delete()
50         .eq('id', id);
51
52   1x    if (error) {
53     1x     console.error('DELETE ERROR:', error);
54   }
55 }
56
57 // SEED (optional)
58 async function seed() {
59   8x   const bcrypt = require('bcryptjs');
60
61   8x   const users = await read('users');
62
63   8x   if (users.length === 0) {
64     6x     const { v4: uuidv4 } = require('uuid');
65
66     6x     await insert('users', [
67       {
68         id: uuidv4(), //  uuid เท่านั้น
69         username: 'admin',
70         email: 'admin@starwish.com',
71         password: bcrypt.hashSync('admin123', 10),
72         role_id: 1, //  ไม่ role_id
73         created_at: new Date().toISOString() //  snake_case
74       }
75     ]);
76
77     6x     console.log('  Seed users complete');
78   }
79 }
80
81 8x   seed();
82
83 8x   module.exports = { read, insert, update, remove };

```

All files / utils supabase.js

100% Statements 3/3

100% Branches 0/0

100% Functions 0/0

100% Lines 3/3

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```

1 3x const { createClient } = require('@supabase/supabase-js');
2
3  /**
4   * Shared Supabase client used by server-side routes.
5   *
6   * Required env:
7   * - `SUPABASE_URL`
8   * - `SUPABASE_KEY`
9   *
10  * @type {Object}
11  */
12 3x const supabase = createClient(
13    process.env.SUPABASE_URL,
14    process.env.SUPABASE_KEY
15  );
16
17 3x module.exports = supabase;

```

จากการรันทดสอบระบบ (Test Cases) พบว่าโค้ดบรรลุเป้าหมาย Statement Coverage ที่ 100% ซึ่งเป็นเครื่องยืนยันว่าชุดคำสั่งทุกบรรทัดได้รับการประมวลผลอย่างสมบูรณ์ โดยเฉพาะไฟล์จัดการข้อมูลหลัก ได้แก่ `answerModel.js`, `taskModel.js` และ `userModel.js` ที่ผ่านการทดสอบครอบคลุมทุกเงื่อนไข ผลลัพธ์ที่ได้นี้ช่วยรับประกันว่าระบบมีโครงสร้างที่รัดกุม ทำงานได้อย่างถูกต้องแม่นยำ และพร้อมตอบสนองการใช้งานตามที่คาดหวังไว้ได้อย่างเต็มประสิทธิภาพ

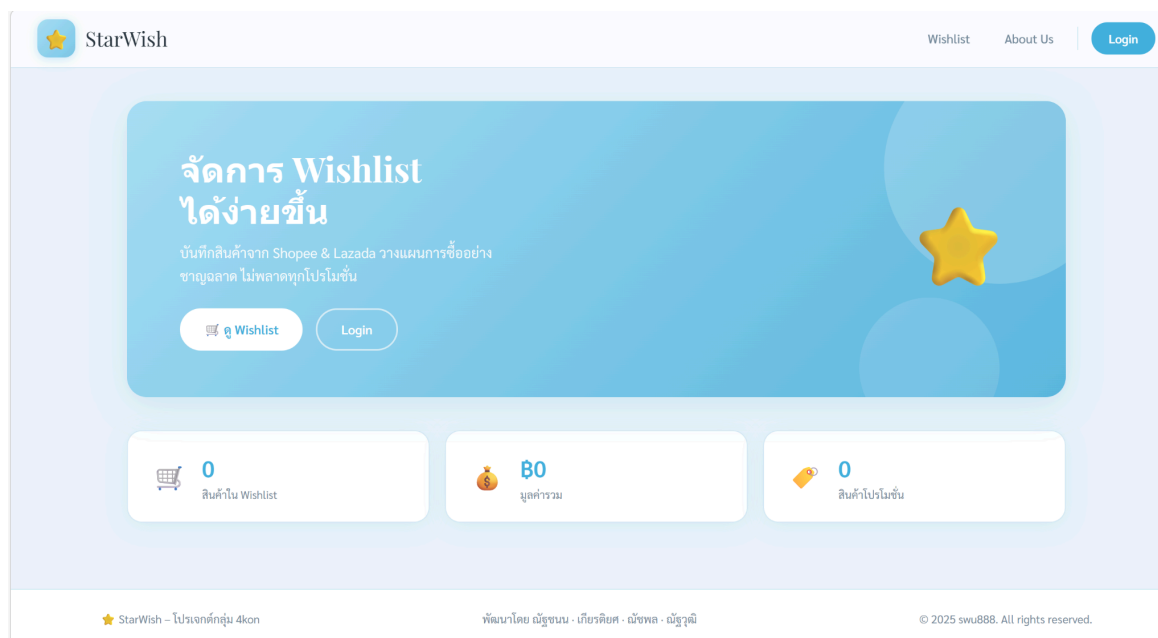
5. สิ่งที่ยังไม่สมบูรณ์ (Bugs / Errors)

- Bug: พบปัญหาการจัดการ Session แม้จะ logout session ก็ยังคงถูกเก็บไว้
- Bug: user ลบ categories ไม่ได้
- Bug: filter การ sort ราคาสินค้าไม่ทำงาน
- Error: การจัดการ Error Handling ในกรณีที่ Supabase API ตอบสนองช้าเกินไป (Timeout) ยังไม่ได้ทำ Circuit Breaker

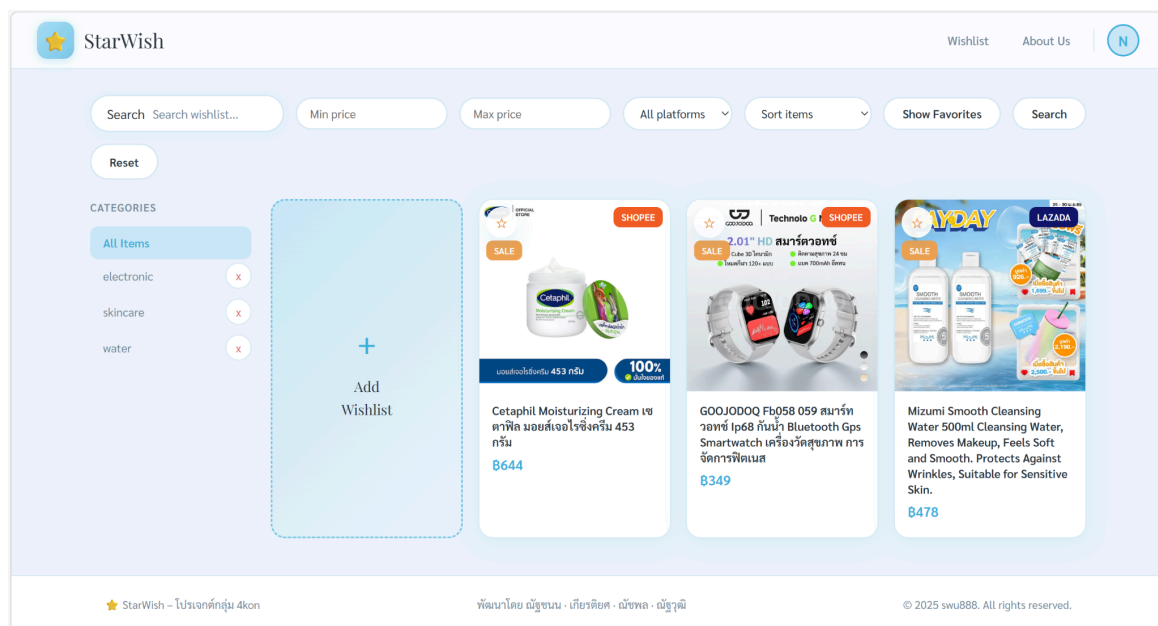
Website UI Screenshots & Mockups

จากข้อมูล Figma และโครงสร้างระบบ ท่านมี:

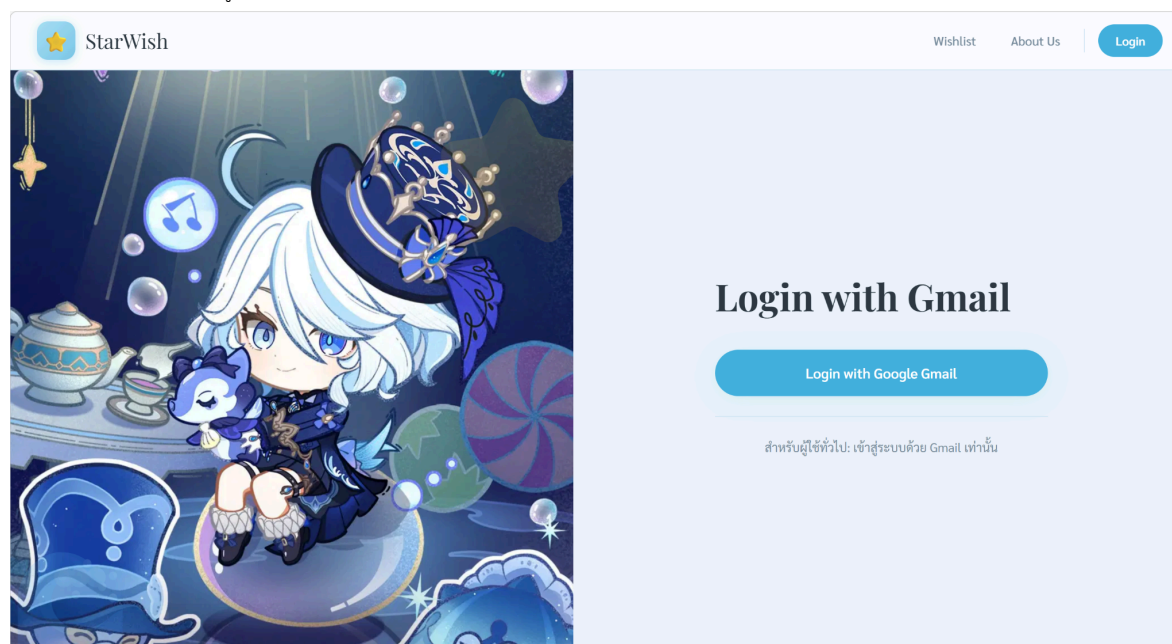
- Main Page - หน้าแรกแสดงรายการสินค้า



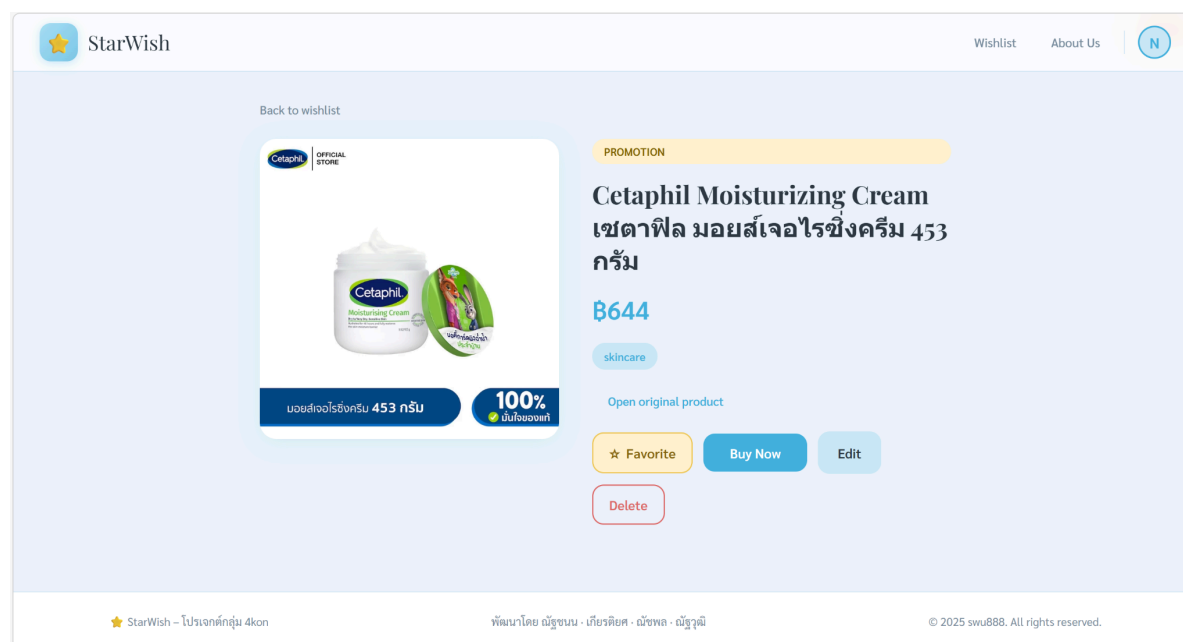
- หน้า wishlist



- Login Page - ระบบเข้าสู่ระบบ



- Product Detail - รายละเอียดสินค้า



- Add product


StarWish
Wishlist
About Us
N

... กลับ

+ เพิ่มสินค้าใหม่



URL รูปภาพสินค้า

ชื่อสินค้า *

ราคา (บาท) *

แพลตฟอร์ม

TAGS

เพิ่ม tag เช่น แท็ก แกรนด์ +

มี tag 1 tag

ลิงก์สินค้า

รายละเอียด

☐ สินค้าไปรษณีย์

ยกเลิก

บันทึก

- About Us


StarWish
Wishlist
About Us
Login

About StarWish

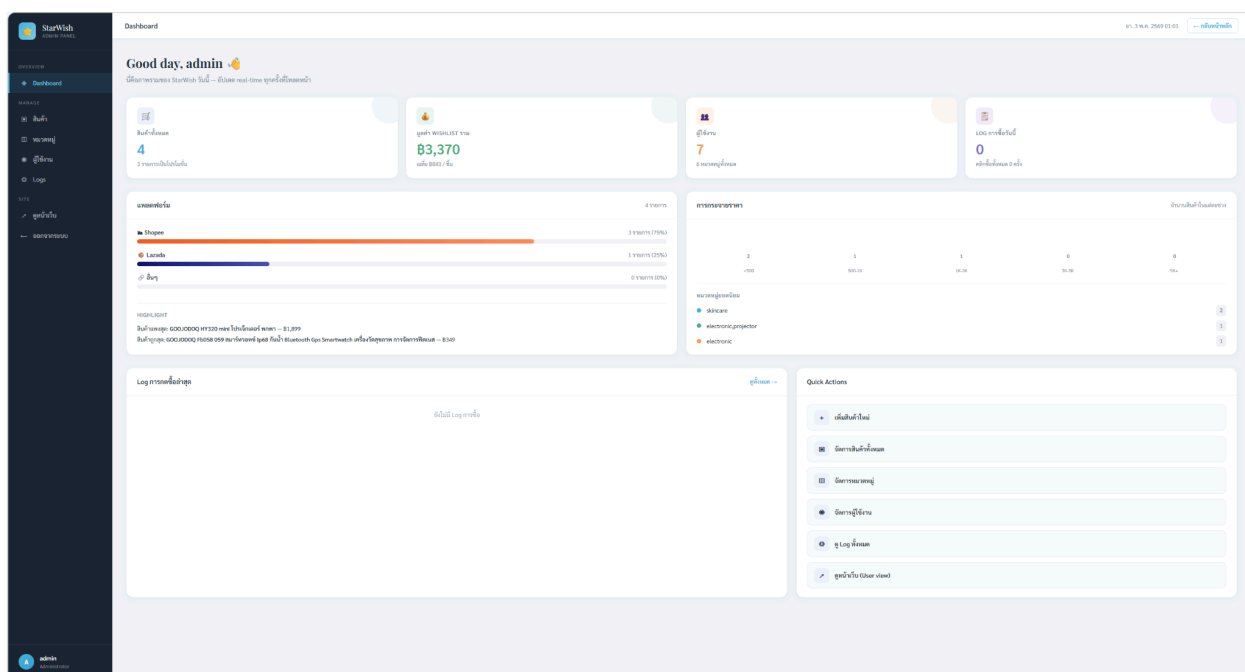
Turn your wishes into reality ✨

StarWish is a simple and beautiful wishlist platform that helps you organize the things you love, track your dreams, and share them with others. Whether it's a gift, a goal, or inspiration — every wish matters.

How to use

You can save products from various platforms like Shopee and Lazada, making it easier to compare and decide before buying. Just copy the product link and paste it into StarWish to add it to your wishlist. You can also organize your wishes into different lists, set priorities, and share them with friends and family. StarWish is designed to be your personal wish organizer, making it easier to keep track of everything you want in one place.

- Admin Dashboard - จัดการสินค้าและคุณสมบัติ



StarWish ADMIN PANEL

จัดการสินค้า

4 รายการทั้งหมด

3 พ.ค. 2569 01:01

กลับหน้าหลัก

+ เพิ่มสินค้า

ชื่อสินค้า	ราคา	หมวดหมู่	แพลตฟอร์ม	โปรโมชั่น	วันที่เพิ่ม	จัดการ
GOOJODOQ HY320 mini โปรเจกเตอร์ พกพา	฿1,899	electronic,projector	SHOPEE	-	24/4/2569	 
Cetaphil Moisturizing Cream เซตาฟิล มอยส์เจอไรเซอร์ครีม 453 กรัม	฿644	skincare	SHOPEE	PROMO	29/4/2569	 
GOOJODOQ Fb058 059 สมาร์ทวอตช์ Ip68 กันน้ำ Bluetooth Gps Smartwatch เครื่องวัดสุขภาพ การจัดการฟิตเนส	฿349	electronic	SHOPEE	PROMO	29/4/2569	 
Mizumi Smooth Cleansing Water 500ml Cleansing Water, Removes Makeup, Feels Soft and Smooth. Protects Against Wrinkles, Suitable for Sensitive Skin.	฿478	skincare	LAZADA	PROMO	29/4/2569	 

StarWish

ADMIN PANEL

OVERVIEW

Dashboard

MANAGE

สินค้า

หมวดหมู่

ผู้ใช้งาน

Logs

SITE

ดูหน้าเว็บ

ออกจากระบบ

admin

Administrator

จัดการหมวดหมู่

ธ. 3 พ.ค. 2569 01:03

← กลับหน้าหลัก

จัดการหมวดหมู่

6 หมวดหมู่

เพิ่มหมวดหมู่ใหม่

+ เพิ่ม

ชื่อหมวดหมู่	จำนวนสินค้า	จัดการ
electronic	1 รายการ	
skincare	2 รายการ	
tranformer	0 รายการ	
water	0 รายการ	
projector	0 รายการ	
glasses	0 รายการ	

StarWish

ADMIN PANEL

OVERVIEW

Dashboard

MANAGE

สินค้า

หมวดหมู่

ผู้ใช้งาน

Logs

SITE

ดูหน้าเว็บ

ออกจากระบบ

admin

Administrator

จัดการผู้ใช้งาน

ธ. 3 พ.ค. 2569 01:02

← กลับหน้าหลัก

จัดการผู้ใช้งาน

7 บัญชีทั้งหมด

USERNAME	EMAIL	ROLE	สมัครเมื่อ	จัดการ
admin	admin@starwish.com	USER	15/4/2569	(ดู)
Nuttavut Tochue	nuttavut.tochue@g.swu.ac.th	USER	23/4/2569	
Natchanon Marlasin	scoldingx2@gmail.com	USER	23/4/2569	
SNOWFLAKE	swanergamer@gmail.com	USER	23/4/2569	
Natchanon Marlasin7590	natchanon.marlasin@g.swu.ac.th	USER	23/4/2569	
Arkroll	arkroll088@gmail.com	USER	23/4/2569	
Nuttavut Tochue2113	nuttavut.tochue@gmail.com	USER	24/4/2569	

UI Test Cases (5 Test Cases)

ลำดับ	หัวข้อ / ฟังก์ชัน (Feature)	การกระทำ / เงื่อนไข (Action / Condition)	ผลลัพธ์ที่คาดหวัง (Expected Result)
1	หน้า Wishlist แสดงสินค้าตาม Platform	เลือกกรอง (Filter) เฉพาะ Shopee หรือ Lazada	ต้องแสดงเฉพาะสินค้าที่ตรงกับ Platform ที่เลือกไว้เท่านั้น
2	หน้า Wishlist ค้นหาสินค้าได้	พิมพ์ Keyword ลงในช่องค้นหา (Search)	ต้องแสดงเฉพาะสินค้าที่มีชื่อหรือคำอธิบายตรงกับ Keyword
3	หน้า Wishlist เรียงราคาจากน้อยไปมาก/ มากไปน้อย	เลือกการจัดเรียง (Sort) แบบ price-asc/desc	สินค้าต้องเรียงลำดับจากราคา ต่ำไปสูง/สูงไปต่ำ
4	กดปุ่ม Google Login	คลิกปุ่ม Login with Google Gmail	ต้อง redirect ไปยังเส้นทาง /auth/google?redirect=...
5	OAuth callback กลับมาที่ระบบ	หลัง login ผ่าน Google แล้ว กลับมาที่ระบบ	ต้องกลับไปยังหน้า login ด้วย redirect ที่ถูกต้อง และพร้อมทำขั้นตอน sync user

3. Profiling Report (Static & Dynamic)

เปรียบเทียบกับ Phase 3:

- Static Profiling:

1. ตารางสรุปภาพรวม (Overall Summary)

รายการ (Metric)	Phase 3	Phase 4	ผลต่าง (Difference)
Total SLOC (จำนวนบรรทัดรวม)	576	2,100	+1,524 ▲
Average SLOC (ค่าเฉลี่ยบรรทัด/ไฟล์)	64	161	+97 ▲
Total Maintainability	648.00	919.686	+271.679 ▲
Average Maintainability	72.00	70.75	-1.25 ▼
Total JSHint Messages	146	328	+182 ▲

ข้อสังเกตภาพรวม:

- Phase 4 มีปริมาณ SLOC เพิ่มขึ้นอย่างก้าวกระโดด เนื่องจากการเพิ่มไฟล์ทดสอบ (Test Files), ไฟล์วิเคราะห์ (Profiling Files) และไฟล์ประกอบอื่นๆ
- ค่า Maintainability เฉลี่ยลดลงเล็กน้อย แต่ยังคงอยู่ในระดับที่สามารถบำรุงรักษาและใช้งานได้ดี
- จำนวน JSHint Messages เพิ่มขึ้น ซึ่งเป็นผลแปรผันตรงตามจำนวนไฟล์และบรรทัดโค้ดที่ถูกวินิจฉัยมากขึ้น

2. ตารางเปรียบเทียบไฟล์ที่มีอยู่ทั้งสอง Phase (File Comparison)

file	SLOC	Maintainability	JSHint Messages
	P.3 → P.4 (ผลต่าง)	P.3 → P.4 (ผลต่าง)	P.3 → P.4 (ผลต่าง)
app.js	48 → 72 (+24)	86.965 → 75.997 (-10.968)	14 → 20 (+6)
middleware/auth.js	15 → 40 (+25)	74.819 → 76.924 (+2.105)	2 → 2 (0)
routes/categories.js	38 → 282 (+244)	71.675 → 64.349 (-7.326)	20 → 35 (+15)

ข้อสังเกตรายไฟล์:

- **app.js**: มีขนาดบรรทัดเพิ่มขึ้นและค่าความดูแลรักษาลดลง เนื่องจากการเพิ่ม Logic และ Dependency ที่มากขึ้น
- **middleware/auth.js**: แม้จะมีการเขียนโค้ดเพิ่มขึ้น แต่ค่า Maintainability กลับปรับตัวดีขึ้นเล็กน้อย
- **routes/categories.js**: เป็นไฟล์ที่มีความซับซ้อนเพิ่มขึ้นมากที่สุดใน Phase 4 เนื่องจากการรวมไฟล์ทดสอบและการอ้างอิงที่เกี่ยวข้องจำนวนมาก

3. ตารางไฟล์ใหม่ที่เพิ่มเข้ามาใน Phase 4 (New Files Added)

New Files	SLOC	Maintainability	JSHint

jest.config.js	48	81.009	0
js/user auth.js	53	62.812	1
public/js/admin.js	42	78.503	9

ข้อสังเกตรายละเอียดไฟล์:

- **app.js**: มีขนาดบรรทัดเพิ่มขึ้นและค่าความดูแลรักษาลดลง เนื่องจากการเพิ่ม Logic และ Dependency ที่มากขึ้น
- **middleware/auth.js**: แม้จะมีการเขียนโค้ดเพิ่มขึ้น แต่ค่า Maintainability กลับปรับตัวดีขึ้นเล็กน้อย
- **routes/categories.js**: เป็นไฟล์ที่มีความซับซ้อนเพิ่มขึ้นมากที่สุดใน Phase 4 เนื่องจากการรวมไฟล์ทดสอบและการอ้างอิงที่เกี่ยวข้องจำนวนมาก

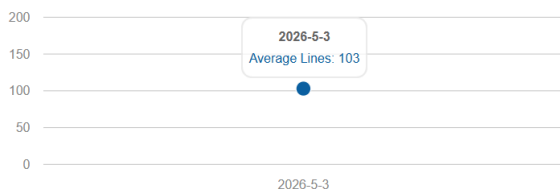
สรุปผลการเปรียบเทียบ (Conclusion)

- เมื่อเปรียบเทียบโครงสร้างระบบระหว่าง Phase 3 และ Phase 4 พบว่าใน Phase 4 มีขนาดของซอร์สโค้ดรวมเพิ่มขึ้นอย่างมีนัยสำคัญ โดยมีสาเหตุหลักมาจากการบูรณาการระบบทดสอบ (Unit Tests), Test Harness และไฟล์ประกอบอื่น ๆ ส่งผลให้จำนวนบรรทัดรวม (Total SLOC) เพิ่มขึ้นจาก 576 เป็น 2,100 บรรทัด ในขณะที่ค่าเฉลี่ยความสามารถในการบำรุงรักษา (Maintainability) ลดลงเพียงเล็กน้อยจาก 72.00 เป็น 70.75 ซึ่งยังคงอยู่ในเกณฑ์มาตรฐานที่ยอมรับและดูแลต่อได้ จุดที่มีการเปลี่ยนแปลงอย่างโดดเด่นคือไฟล์ **routes/categories.js** และ **app.js** ที่มีโครงสร้างความซับซ้อนและลอจิกเพิ่มขึ้น เพื่อรองรับการขยายขีดความสามารถของฟังก์ชันในระบบ

Summary

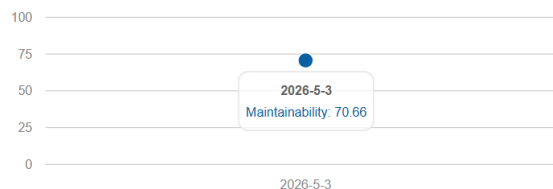
Total/Average Lines ⓘ

826 / 103



Average Maintainability ⓘ

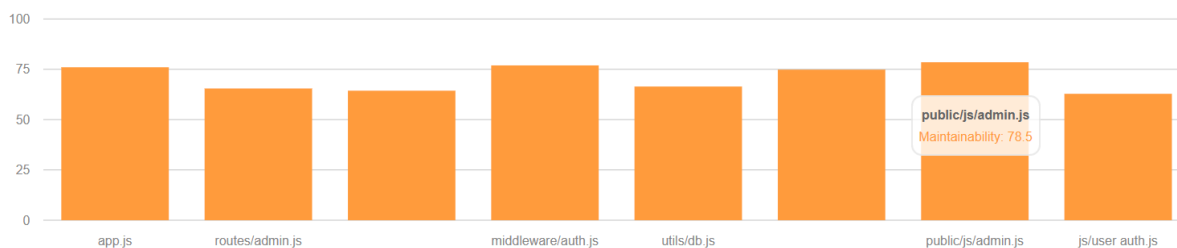
70.66



Static profiling และ Dynamic profiling - Overview

- จำนวนบรรทัดทั้งหมดมีทั้งหมด 826 บรรทัด และมีจำนวนบรรทัดโดยเฉลี่ย 103 บรรทัด ค่าบำรุงรักษาโดยเฉลี่ย เท่ากับ 70.66 ซึ่งแปลว่ายังอยู่ในเกณฑ์ที่ค่อนข้างดี

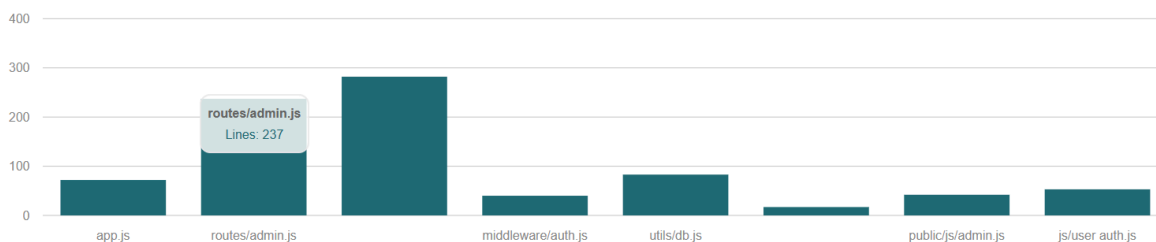
Maintainability ⓘ



ค่าการบำรุงรักษา

- จะเห็นว่าไฟล์ `public/js/admin.js` บำรุงรักษาง่ายที่สุดโดยมีการบำรุงรักษา 78.50 รองลงมาคือ ไฟล์ `middleware/auth.js` มีค่าการบำรุงรักษา 76.92 รองลงมาคือ ไฟล์ `app.js` มีค่าการบำรุงรักษา 75.99 รองลงมาคือ ไฟล์ `utils/supabase.js` มีค่าการบำรุงรักษา 74.83 และลำดับสุดท้ายคือ ไฟล์ `js/user auth.js` มีค่าการบำรุงรักษา 62.81

Lines of code ⓘ



จำนวนบรรทัดในการเขียนโค้ด

- จะเห็นว่าไฟล์ `routes/categories.js` มีจำนวนบรรทัดในการเขียนโค้ด 282 บรรทัด รองลงมาคือ ไฟล์ `routes/admin.js` มีจำนวนบรรทัดในการเขียนโค้ด 237 บรรทัด รองลงมาคือ ไฟล์ `utils/db.js` มีจำนวนบรรทัดในการเขียนโค้ด 83 บรรทัด รองลงมาคือ ไฟล์ `app.js` มีจำนวนบรรทัดในการเขียนโค้ด 72 บรรทัด และลำดับสุดท้าย คือไฟล์ `utils/supabase.js` มีจำนวนบรรทัดในการเขียนโค้ด 17 บรรทัด

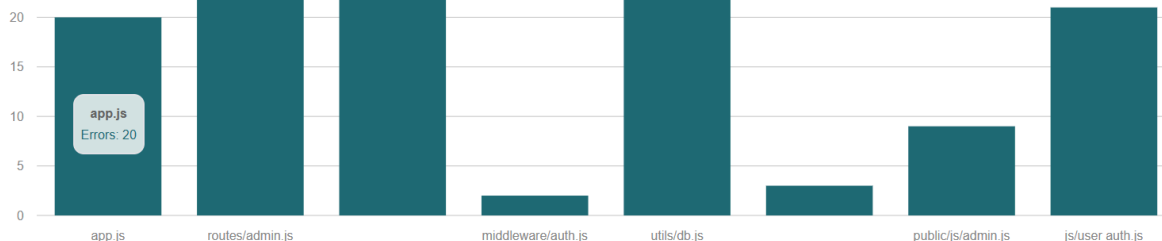
Estimated errors in implementation ⓘ



ข้อผิดพลาดโดยประมาณในการใช้งาน

- จะเห็นว่าไฟล์ `routes/admin.js` มีข้อผิดพลาดโดยประมาณในการใช้งาน 2.93 รองลงมาคือ ไฟล์ `routes/categories.js` มีข้อผิดพลาดโดยประมาณในการใช้งาน 2.78 รองลงมาคือ ไฟล์ `app.js` มีข้อผิดพลาดโดยประมาณในการใช้งาน 0.69 รองลงมาคือ ไฟล์ `js/user auth.js` มีข้อผิดพลาดโดยประมาณในการใช้งาน 0.60 และลำดับสุดท้ายคือ ไฟล์ `utils/supabase.js` มีข้อผิดพลาดโดยประมาณในการใช้งาน 0.04 และลำดับสุดท้ายคือ ไฟล์ `jest.config.js` มีข้อผิดพลาดโดยประมาณในการใช้งาน 0.01

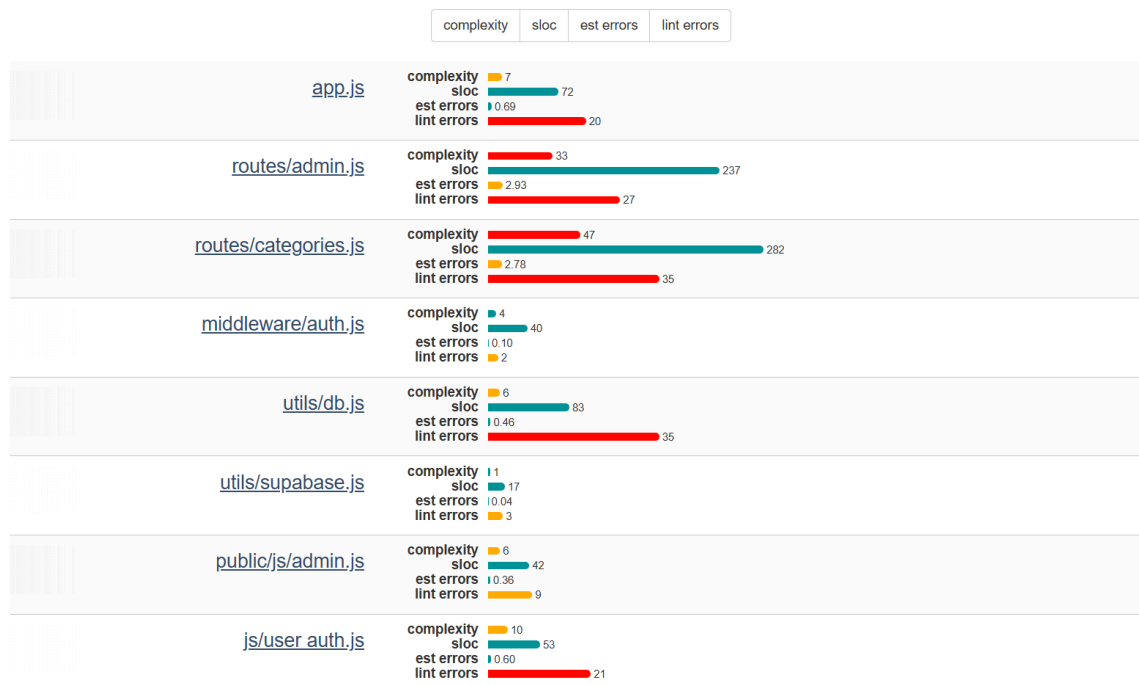
Lint errors



การตรวจสอบ code จาก linter ข้อผิดพลาดหรือคำเตือนเรื่องมาตรฐานหรือคุณภาพของโค้ด โอกาสความผิดพลาดแค่ไหนโดยที่เราไม่ต้องมา run web

- จะเห็นว่าไฟล์ `routes/categories.js` และ `utils/db.js` มีข้อผิดพลาดโดยประมาณในการใช้งาน 35 รองลงมาคือ ไฟล์ `routes/admin.js` มีข้อผิดพลาดโดยประมาณในการใช้งาน 27 รองลงมาคือ ไฟล์ `js/user auth.js` มีข้อผิดพลาดโดยประมาณในการใช้งาน 21 รองลงมาคือ ไฟล์ `app.js` มีข้อผิดพลาดโดยประมาณในการใช้งาน 20 และลำดับสุดท้ายคือ ไฟล์ `middleware/auth.js` มีข้อผิดพลาดโดยประมาณในการใช้งาน 2

Files



- complexity ไฟล์ `routes/categories.js` มีความซับซ้อนมากกว่าไฟล์ `routes/admin.js`, `js/user_auth.js`, `app.js` และ `utils/supabase.js`
- SLOC (source line of code) ไฟล์ `routes/categories.js` มีจำนวนบรรทัดโค้ดทั้งหมด 282 บรรทัด , ไฟล์ `routes/admin.js` มีจำนวนบรรทัดโค้ดทั้งหมด 237 บรรทัด , ไฟล์ `utils/db.js` มีจำนวนบรรทัดโค้ดทั้งหมด 83 บรรทัด , ไฟล์ `app.js` มีจำนวนบรรทัดโค้ดทั้งหมด 72 บรรทัด และไฟล์ `utils/supabase.js` มีจำนวนบรรทัดโค้ดทั้งหมด 17 บรรทัด
- est errors ข้อผิดพลาดโดยประมาณในการใช้งาน ไฟล์ `routes/admin.js` 2.93 , `routes/categories.js` 2.78 , `app.js` 0.69 , `js/user_auth.js` 0.60 และไฟล์ `utils/supabase.js` 0.04
- lint error ตรวจสอบ code ว่ามีโอกาสมิติดพลาดแค่ไหน ไฟล์ `routes/categories.js` และ `utils/db.js` 35 , `routes/admin.js` 27 , `js/user_auth.js` 21 , `app.js` 20 และไฟล์ `middleware/auth.js` 2

app.js

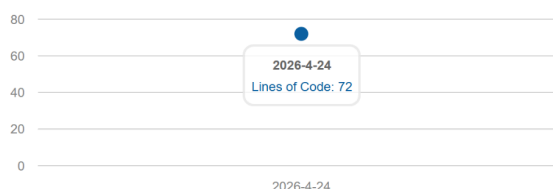
Maintainability ⓘ

76.00



Lines of code ⓘ

72



Difficulty ⓘ

15.37

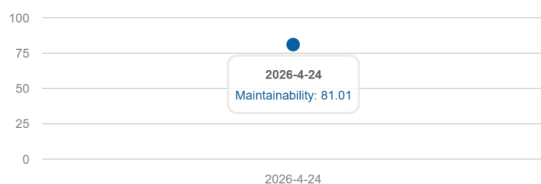
Estimated Errors ⓘ

0.69

jest.config.js

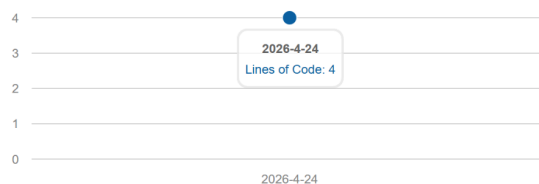
Maintainability ⓘ

81.01



Lines of code ⓘ

4



Difficulty ⓘ

2.00

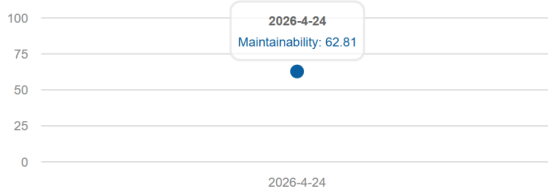
Estimated Errors ⓘ

0.01

js/user auth.js

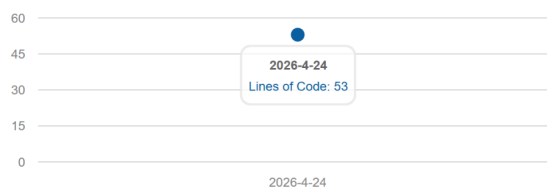
Maintainability ⓘ

62.81



Lines of code ⓘ

53



Difficulty ⓘ

21.06

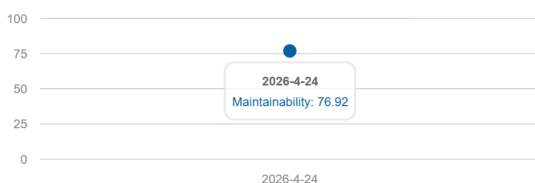
Estimated Errors ⓘ

0.60

middleware/auth.js

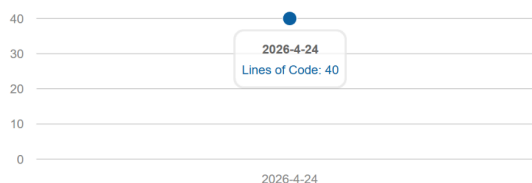
Maintainability ⓘ

76.92



Lines of code ⓘ

40



Difficulty ⓘ

12.38

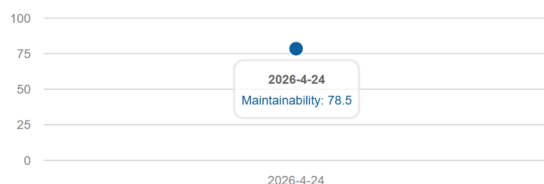
Estimated Errors ⓘ

0.10

public/js/admin.js

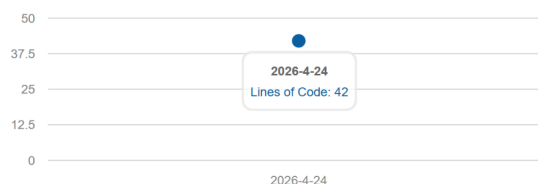
Maintainability ⓘ

78.50



Lines of code ⓘ

42



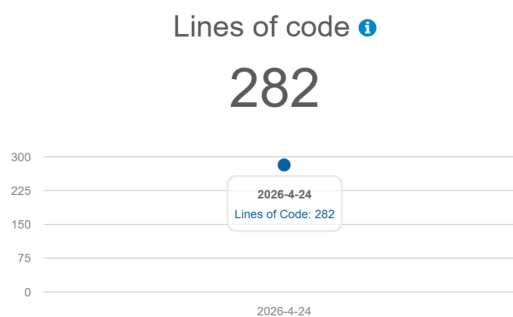
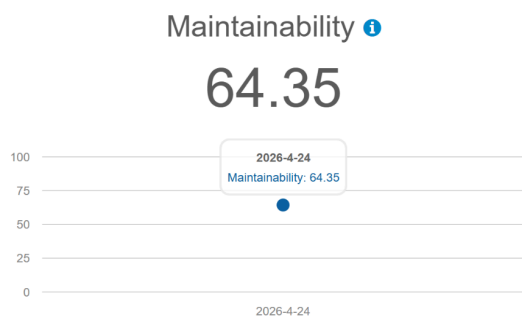
Difficulty ⓘ

12.15

Estimated Errors ⓘ

0.36

routes/categories.js



Difficulty ⓘ

54.19

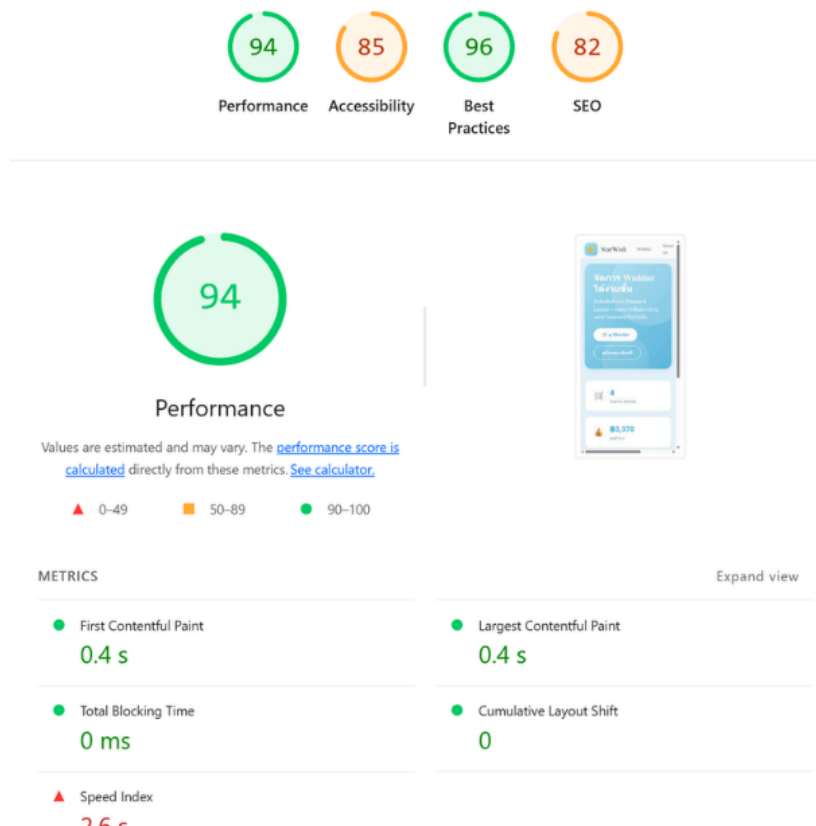
Estimated Errors ⓘ

2.78

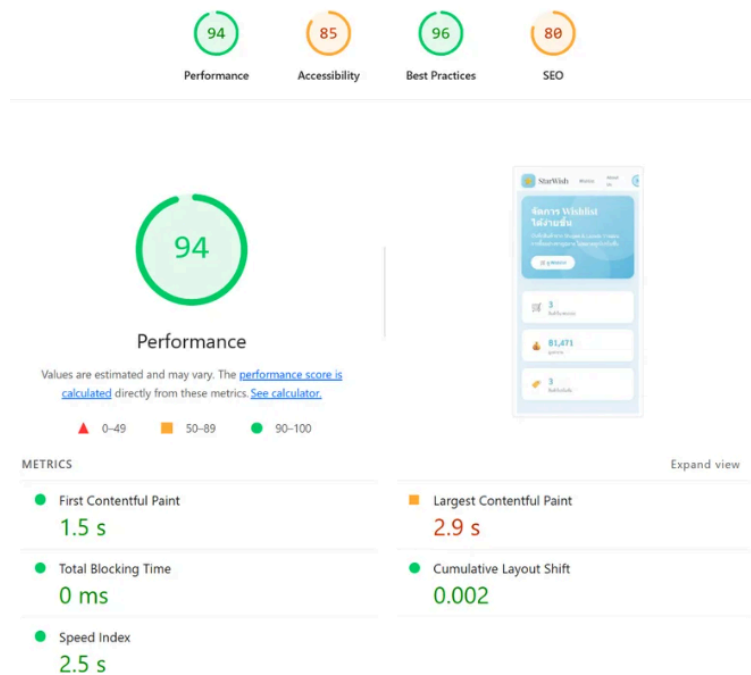
- Dynamic Profiling: lighthouse

Home page

Phase 3:

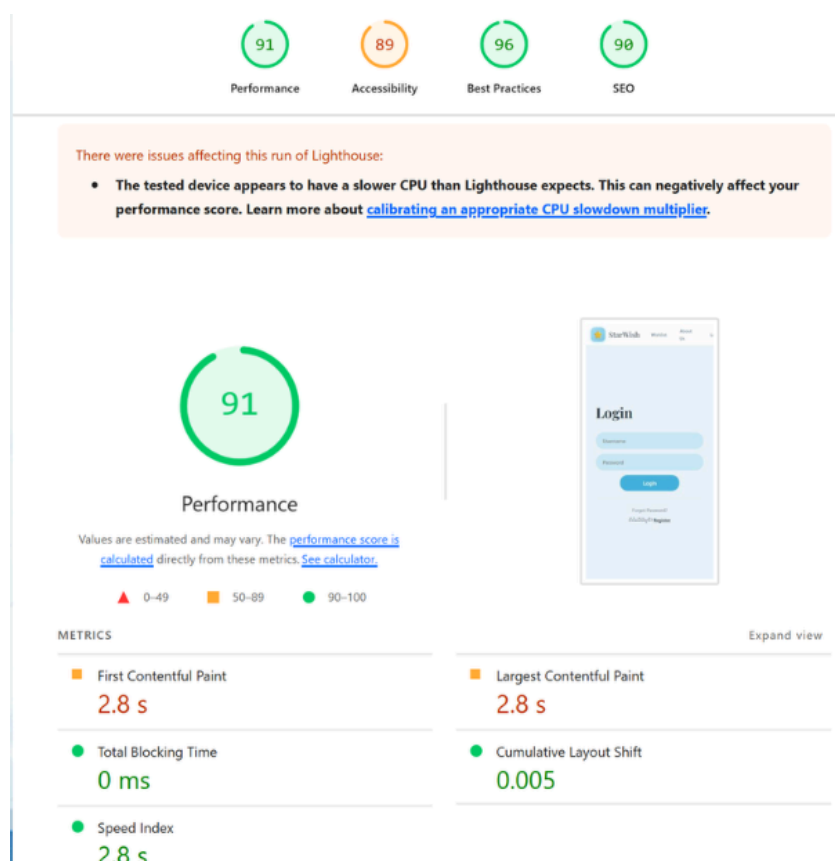


Phase 4:

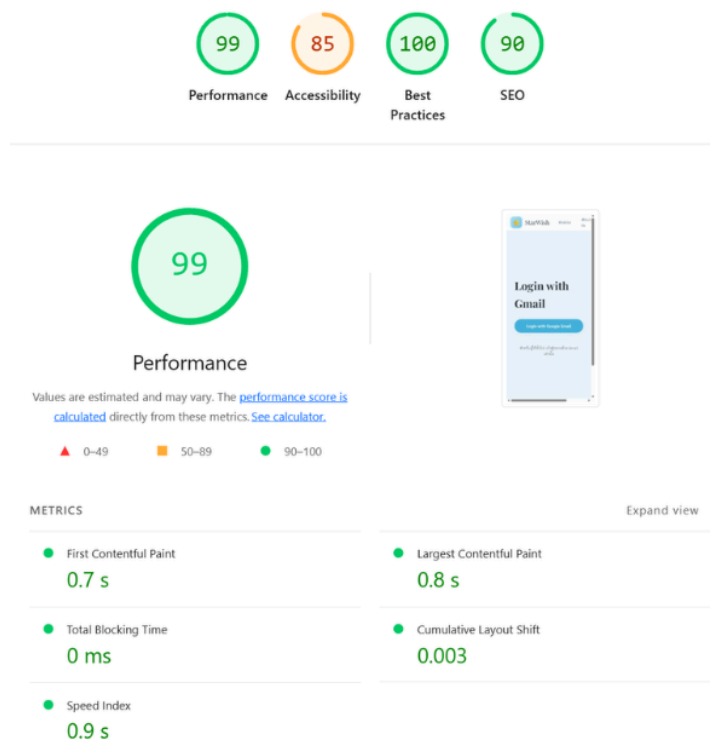


Login

Phase 3:



Phase 4:



4. CI/CD Pipeline Explanation

อธิบายการทำ CI/CD ที่ใช้ในการทำ product

โปรเจกต์นี้ใช้ GitHub Actions เป็นเครื่องมือหลักในการทำ CI/CD โดยตั้ง workflow ให้รันอัตโนมัติเมื่อมีการ push หรือ pull request ไปยัง branch main จากภาพผลการทำงานของ GitHub Actions จะเห็นว่า workflow ต่าง ๆ เช่น Node.js CI, security-audit.yml และ semgrep ถูกเรียกใช้งานและแสดงสถานะผ่านทั้งหมด ซึ่งช่วยให้ทีมมั่นใจได้ว่าโค้ดที่ส่งขึ้น repository ผ่านการตรวจสอบแล้วก่อนนำไปใช้งานจริง

ขั้นตอนการทำ CI/CD ด้วย GitHub

1. สร้างไฟล์ workflow ภายในโฟลเดอร์ .github/workflows/
2. กำหนดให้ workflow ทำงานเมื่อมี push หรือ pull_request
3. กำหนด job สำหรับรัน test, build, และตรวจสอบความปลอดภัยของโค้ด
4. ใช้ actions/checkout@v4 เพื่อดึง source code จาก repository
5. ใช้ actions/setup-node@v4 เพื่อติดตั้ง Node.js และ cache npm

6. รัน npm ci เพื่อติดตั้ง dependencies แบบสะอาดและได้ผลลัพธ์คงที่
7. รัน npm test เพื่อทดสอบระบบอัตโนมัติ
8. หากมี build script ให้รัน npm run build --if-present
9. ตรวจสอบผลลัพธ์จาก GitHub Actions ว่าผ่านหรือไม่
10. เมื่อ CI ผ่านแล้วจึงนำโค้ดไป deploy ต่อในขั้น CD

อธิบายการทำงานของ CI/CD

CI/CD คือแนวทางการพัฒนาและส่งมอบซอฟต์แวร์แบบอัตโนมัติ โดยแบ่งออกเป็น 2 ส่วนหลักคือ

- Continuous Integration (CI) คือการนำโค้ดที่นักพัฒนาส่งเข้ามารวมกับโค้ดหลักอย่างสม่ำเสมอ พร้อมตรวจสอบด้วย test และ build อัตโนมัติ
- Continuous Deployment (CD) คือการนำโค้ดที่ผ่านการตรวจสอบแล้วไป deploy สู่ระบบจริงหรือสภาพแวดล้อมที่ใช้งานได้โดยอัตโนมัติ หรือกึ่งอัตโนมัติ

ประโยชน์ของ CI/CD คือช่วยลดข้อผิดพลาดจากการรวมโค้ด, ตรวจสอบ bug ได้เร็ว, เพิ่มความมั่นใจในคุณภาพของระบบ, และช่วยให้ทีมทำงานร่วมกันได้อย่างมีประสิทธิภาพมากขึ้น

สิ่งที่ทำในขั้นตอน Continuous Integration (CI)

ในขั้นตอน CI ของโปรเจกต์นี้ มีการทำงานดังนี้

- ดึง source code ล่าสุดจาก GitHub
- ติดตั้ง dependencies ด้วย npm ci
- รัน unit test ด้วย npm test
- ตรวจสอบการ build ของโปรเจกต์
- ใช้ matrix strategy เพื่อทดสอบหลายเวอร์ชันของ Node.js เช่น 18.x, 20.x และ 22.x
- ทำ security check เพิ่มเติมผ่าน workflow เช่น security-audit.yml และ semgrep

ขั้นตอนเหล่านี้ช่วยให้มั่นใจว่าโค้ดที่ merge เข้ามาไม่มีปัญหาเรื่อง syntax, test fail, หรือ dependency ที่เสี่ยงต่อความปลอดภัย

สิ่งที่ทำในขั้นตอน Continuous Deployment (CD)

ในขั้นตอน CD ของโปรเจกต์นี้ มีการทำงานดังนี้

- หลังจากโค้ดผ่าน CI แล้ว Render จะนำโค้ดไป deploy อัตโนมัติ
- Render ทำหน้าที่เป็น hosting ที่ใช้ระบบจริง
- เมื่อมีการอัปเดตโค้ดใหม่บน branch ที่เชื่อมต่อไว้ ระบบจะ deploy เวอร์ชันล่าสุดให้โดยอัตโนมัติ
- ช่วยให้ผู้ใช้เข้าถึงเว็บเวอร์ชันล่าสุดได้ตลอดเวลา

กระบวนการพัฒนา Software ของโครงการ 4kon

กระบวนการทำงานใน Sprint ที่ 1

ในการพัฒนา Software สำหรับโครงการ 4kon ทีมพัฒนาได้เลือกใช้แนวทาง Agile Process Model เพื่อให้การทำงานมีความยืดหยุ่นและสามารถปรับปรุงได้อย่างต่อเนื่อง ใน Sprint ที่ 1 ทีมงานเริ่มต้นด้วยการเก็บ Requirement จากความต้องการของผู้ใช้และการวิเคราะห์ลักษณะของระบบที่ต้องการพัฒนา จากนั้นจึงทำการแยก Requirement ออกเป็นส่วนย่อย ๆ และแบ่งเป็น Functional Requirement และ Non-Functional Requirement เพื่อให้สามารถมองเห็นขอบเขตของระบบได้ชัดเจนมากขึ้น

หลังจากนั้นทีมพัฒนาได้จัดลำดับความสำคัญของ Requirement โดยเน้นฟังก์ชันพื้นฐานที่จำเป็นต่อการใช้งานก่อน เช่น หน้าหลัก, หน้าแสดงสินค้า, หน้า Login, หน้า Register และหน้า About Us รวมถึงเริ่มวางโครงสร้างของระบบและออกแบบหน้าจอด้วย Figma เพื่อใช้เป็นแนวทางในการพัฒนาหน้าเว็บจริงในขั้นตอนต่อไป นอกจากนี้ยังมีการกำหนดแนวทางการแบ่งงานภายในทีมเพื่อให้แต่ละคนรับผิดชอบส่วนที่ชัดเจนและสามารถติดตามความคืบหน้าได้ง่าย

กระบวนการทำงานใน Sprint ที่ 2

ในการพัฒนา Software สำหรับโครงการ 4kon ในช่วง Sprint ที่ 2 ทีมพัฒนาได้ยังคงใช้หลักการของ Agile Process Model เป็นแนวทางหลัก แต่มีการทำงานที่เป็นระบบและสม่ำเสมอมากขึ้น โดยมีการสื่อสารกันผ่าน Discord และใช้ GitHub ในการจัดการซอร์สโค้ดและติดตามการเปลี่ยนแปลงของงานอย่างต่อเนื่อง

ใน Sprint นี้ ทีมพัฒนาเริ่มลงมือพัฒนาโครงสร้างหน้าเว็บจริง เช่น หน้า Home, หน้า Wishlist, หน้า Product Detail, หน้า Login และหน้า Register รวมถึงปรับปรุง UI ให้สอดคล้องกับแบบที่ออกแบบไว้ใน Figma นอกจากนี้ยังเริ่มเตรียมโครงสร้างการเชื่อมต่อกับฐานข้อมูลและระบบจัดการข้อมูลสินค้าผ่าน Supabase เพื่อรองรับการแสดงผลรายการสินค้า การค้นหา การกรองข้อมูล และการจัดหมวดหมู่สินค้า

นอกจากนี้ ทีมพัฒนายังใช้ Figma ในการปรับแต่งหน้าตาของระบบให้เหมาะกับการใช้งานจริง และใช้ Mermaid.js ในการวาดแผนภาพ Use Case Diagram เพื่ออธิบายภาพรวมของระบบ โดย Sprint นี้ถือเป็นช่วงสำคัญที่ช่วยเปลี่ยนจากงานออกแบบไปสู่การพัฒนาโค้ดจริง

กระบวนการทำงานใน Sprint ที่ 3

ในการพัฒนา Software สำหรับโครงการ 4kon ในช่วง Sprint ที่ 3 ทีมพัฒนาเน้นไปที่การพัฒนาโค้ดจริงและการทดสอบระบบ เพื่อให้แน่ใจว่าฟังก์ชันหลักของโปรเจกต์สามารถทำงานได้อย่างถูกต้องและมีประสิทธิภาพก่อนนำไปใช้งานจริง

การพัฒนาและเขียนโค้ด (Implementation)

ทีมพัฒนาเริ่มจากการนำ Requirement ที่ผ่านการจัดลำดับความสำคัญแล้วมาพัฒนาเป็นฟีเจอร์จริง โดยใช้ Git และ GitHub ในการจัดการเวอร์ชันของโค้ดและแบ่งงานเป็น branch ย่อยเพื่อป้องกันความขัดแย้งของโค้ดในทีม มีการใช้ Pull Request และการตรวจสอบโค้ดก่อน merge เข้าสู่ branch หลัก

ในส่วนของฟังก์ชันหลัก ทีมพัฒนาได้สร้างระบบจัดการสินค้า, ระบบหมวดหมู่, ระบบผู้ใช้, หน้า Admin Dashboard และระบบ Authentication ที่ใช้ Google OAuth เป็นหลัก รวมถึงพัฒนา route ต่าง ๆ สำหรับการแสดงข้อมูลสินค้า การกรองสินค้า และการบันทึก log การกดซื้อสินค้า

การทดสอบระบบ (Testing)

ทีมพัฒนาเริ่มเขียน Unit Test ด้วย Jest และ Supertest เพื่อทดสอบการทำงานของ route และ utility functions ต่าง ๆ เช่น การ login, register, การเพิ่ม/ลบสินค้า, การเพิ่ม/ลบหมวดหมู่ และการแสดงผลของหน้าเว็บ นอกจากนี้ยังเริ่มสร้างรายงาน coverage เพื่อประเมินว่าชุดทดสอบครอบคลุมโค้ดส่วนสำคัญมากน้อยเพียงใด

กระบวนการทำงานใน Sprint ที่ 4

ใน Sprint ที่ 4 ทีมพัฒนาได้ดำเนินการปรับปรุงและแก้ไขข้อผิดพลาดที่พบจาก Sprint ก่อนหน้า รวมถึงเพิ่มรายละเอียดในการทดสอบและวิเคราะห์ประสิทธิภาพของระบบให้สมบูรณ์มากขึ้น โดยเฉพาะการเพิ่ม UI Testcases พร้อมระบุ Expected Results อย่างชัดเจน เพื่อให้สามารถตรวจสอบการทำงานของระบบได้อย่างเป็นระบบ

นอกจากนี้ ทีมพัฒนาได้ปรับปรุง Static Profiling และ Dynamic Profiling เพื่อใช้วิเคราะห์คุณภาพของโค้ดและเปรียบเทียบผลกับ Sprint ก่อนหน้า รวมถึงปรับปรุง unit tests ให้ครอบคลุมฟังก์ชันสำคัญมากขึ้น เพื่อให้ coverage ของระบบถึงเกณฑ์ที่กำหนดไว้

ในด้านการ Deployment ระบบ ทีมพัฒนาได้นำโปรเจกต์ขึ้นใช้งานผ่าน Render ซึ่งทำหน้าที่เป็น hosting สำหรับตัวแอปพลิเคชัน และใช้ GitHub Actions เป็นระบบ CI Pipeline เพื่อช่วยตรวจสอบโค้ดอัตโนมัติทุกครั้งที่มีการ push หรือ pull request เข้าสู่ branch หลัก โดยกระบวนการนี้ช่วยให้มั่นใจว่าโค้ดที่ deploy ออกไปมีคุณภาพและพร้อมใช้งานจริง

สิ่งที่มีการเปลี่ยนแปลงจาก Sprint ที่ 1

หลังจากการพูดคุยและประเมินผลใน Sprint ที่ 1 ทีมพัฒนาได้ข้อสรุปว่าจำเป็นต้องปรับปรุงบางส่วนของ Requirement เพื่อให้ระบบตอบสนองความต้องการของผู้ใช้ได้ดีขึ้น โดยเฉพาะการเพิ่มความสามารถในการค้นหาและจัดการสินค้าที่ชัดเจนมากขึ้น เช่น การแสดงรายการสินค้าตาม platform, การจัดหมวดหมู่สินค้า, การเรียงลำดับราคา และการแสดงรายละเอียดสินค้าในรูปแบบที่เข้าใจง่าย

นอกจากนี้ยังมีการปรับแนวทางการออกแบบหน้าเว็บให้เหมาะกับระบบจริงมากขึ้น โดยคำนึงถึงการใช้งานของผู้ใช้ทั่วไปและผู้ดูแลระบบร่วมกัน ส่งผลให้โครงสร้างของระบบมีความชัดเจนและนำไปพัฒนาต่อได้ง่ายขึ้นใน Sprint ถัดไป

สิ่งที่มีการเปลี่ยนแปลงจาก Sprint ที่ 2

หลังจากการดำเนินงานใน Sprint ที่ผ่านมา ทีมพัฒนาได้มีการพูดคุยและแลกเปลี่ยนความคิดเห็นเกี่ยวกับปัญหาและอุปสรรคที่พบระหว่างการทำงาน เพื่อหาแนวทางในการปรับปรุงกระบวนการทำงานให้มีประสิทธิภาพมากยิ่งขึ้น โดยหนึ่งในประเด็นที่ทีมพบคือ สมาชิกบางส่วนมีภาระงานภายนอกค่อนข้างมาก ส่งผลให้บางช่วงไม่สามารถโฟกัสกับงานภายในทีมได้อย่างเต็มที่ ดังนั้นทีมจึงมีแนวทางในการวางแผนและจัดการตารางเวลาให้เหมาะสมมากขึ้น เพื่อลดผลกระทบต่อการทำงานร่วมกันภายในทีมในอนาคต

นอกจากนี้ ทีมยังพบว่าช่วงระหว่างการพัฒนา ระบบ สมาชิกหลายคนมีการทำงานต่อเนื่องจนติดเป็นประจำ ซึ่งอาจส่งผลต่อประสิทธิภาพในการทำงาน และสุขภาพของสมาชิกในระยะยาว ทีมจึงมีแนวคิดที่จะบริหารเวลาในการทำงานและการพักผ่อนให้เหมาะสมมากยิ่งขึ้น โดยพยายามลดการทำงานในช่วงเวลาพักผ่อนหากไม่ใช้งานเร่งด่วน เพื่อช่วยลดความเหนื่อยล้าและป้องกันภาวะ Burnout ภายในทีม

อีกทั้ง ทีมพัฒนายังพบว่าสมาชิกบางส่วนยังมีความไม่คุ้นเคยกับการใช้งาน Git และคำสั่งที่เกี่ยวข้อง ส่งผลให้เกิดความกังวลในการจัดการ Source Code และการแก้ไขปัญหา Conflict ดังนั้นทีมจึงมีแนวทางในการศึกษาและรวบรวมคำสั่ง Git ที่ใช้งานบ่อย รวมถึงการแบ่งปันความรู้ภายในทีม เพื่อช่วยให้สมาชิกสามารถใช้งานเครื่องมือในการพัฒนาได้อย่างมั่นใจและมีประสิทธิภาพมากยิ่งขึ้น

นอกจากนี้ ทีมยังพบข้อจำกัดด้านอุปกรณ์และสภาพแวดล้อมในการพัฒนา เช่น ประสิทธิภาพของเครื่องคอมพิวเตอร์ที่อาจไม่รองรับการทำงานบางส่วน ได้อย่างเต็มที่ ส่งผลให้การ Build หรือการทดสอบระบบใช้เวลานานขึ้น ดังนั้นทีมจึงมีแนวทางในการปรับปรุงและเพิ่มประสิทธิภาพของอุปกรณ์ รวมถึงหาแนวทางในการปรับแต่งสภาพแวดล้อมการทำงาน เพื่อช่วยให้กระบวนการพัฒนาซอฟต์แวร์สามารถดำเนินไปได้อย่างราบรื่นและมีประสิทธิภาพมากยิ่งขึ้น

สิ่งที่มีการเปลี่ยนแปลงจาก Sprint ที่ 3

หลังจากเสร็จสิ้นการดำเนินงานใน Sprint ที่ 2 ทีมพัฒนาได้มีการประชุมเพื่อวางแผนและเตรียมความพร้อมสำหรับ Sprint ที่ 3 โดยทีมได้ร่วมกัน ทบทวนผลการดำเนินงานจาก Sprint ก่อนหน้า ซึ่งพบว่าหลายส่วนเริ่มมีความชัดเจนและสามารถทำงานร่วมกันได้อย่างมีประสิทธิภาพมากขึ้น ดังนั้น ใน การเริ่มต้น Sprint ที่ 3 ทีมจึงให้ความสำคัญกับการกำหนดเป้าหมายและทิศทางของการพัฒนาให้ชัดเจนตั้งแต่เริ่มต้น เพื่อให้สมาชิกทุกคนสามารถทำ งานไปในแนวทางเดียวกัน และเข้าใจลำดับความสำคัญของงานที่ต้องดำเนินการในรอบการพัฒนา

นอกจากนี้ ทีมพัฒนาได้ร่วมกันจัดลำดับความสำคัญของงาน (Task Prioritization) เพื่อให้สามารถโฟกัสกับฟังก์ชันหรือส่วนที่มีความสำคัญต่อระบบ ก่อน โดยมีการหารือเกี่ยวกับงานที่ควรดำเนินการก่อน-หลัง รวมถึงการประเมินความเร่งด่วนและผลกระทบของแต่ละงาน เพื่อช่วยให้การดำเนินงานใน Sprint เป็นไปอย่างมีประสิทธิภาพมากยิ่งขึ้น

อีกทั้ง ก่อนเริ่มดำเนินงาน ทีมได้มีการตรวจสอบความพร้อมของแต่ละ Task ไม่ว่าจะเป็นข้อมูล Requirement เอกสารที่เกี่ยวข้อง หรือรายละเอียดที่ จำเป็นต่อการพัฒนา เพื่อให้มั่นใจว่าสมาชิกทุกคนมีข้อมูลเพียงพอสำหรับการเริ่มต้นทำงาน และช่วยลดปัญหาการติดขัดหรือการหยุดชะงักระหว่างการพัฒนา

ทีมพัฒนายังได้มีการหารือเกี่ยวกับการกระจายภาระงานให้เหมาะสมกับสมาชิกแต่ละคน โดยพิจารณาจากความสามารถ ความรับผิดชอบ และปริมาณงานที่ได้รับ เพื่อช่วยให้การทำงานมีความสมดุล ลดภาระงานที่มากเกินไปของสมาชิกบางคน และช่วยให้ทีมสามารถดำเนินงานร่วมกันได้อย่างราบรื่นมากยิ่งขึ้น

นอกจากนี้ ทีมยังได้กำหนดแนวทางการทำงานร่วมกัน (Working Agreement) เพิ่มเติม เช่น การอัปเดตความคืบหน้าของงาน การแจ้งปัญหาที่พบระหว่างการพัฒนา และแนวทางการสื่อสารภายในทีม เพื่อช่วยให้การประสานงานมีความต่อเนื่อง ลดความคลาดเคลื่อนในการทำงาน และทำให้กระบวนการพัฒนาซอฟต์แวร์ใน Sprint ที่ 3 มีประสิทธิภาพมากยิ่งขึ้น

สิ่งที่อยากแก้ไขให้ดีขึ้น ในอนาคตข้างหน้า

1. เวลาจะ commit จะต้อง ระบุ Issue ID ในข้อความ Commit ทุกครั้ง เพื่อลดความสับสนในการตรวจสอบและติดตามความคืบหน้าของงาน
2. ปรับปรุงกระบวนการ Code Review จาก project ไม่ได้ assign หน้าที่ตรวจสอบ code ให้ใครเลย ทำให้หลายๆครั้งไม่มีคนตรวจ pull request เลย
3. กระจายภาระงานให้เหมาะสมมากขึ้น ในบางครั้งกลุ่มของพวกเรากระจายงานกันไม่ค่อยดีทำให้ งานล่าช้า และ งานกระจุกตัวช่วยท้ายๆของ sprint